Exact Consistency in Interactive Multi-Location Encodings: Zero-Error Thresholds and Side Information Bounds

Tristan Simas

***Abstract*—We study exact consistency when one latent fact is represented at multiple locations that may be modified over time. Once an observer's transcript leaves a nontrivial ambiguity class, exact recovery becomes a zero-error information problem.**

We formulate the problem as a zero-error resolvability model for modifiable multi-location encodings. The model separates independent locations from derived ones and measures rate by the number of independently writable degrees of freedom. We prove that zero-incoherence capacity is exactly 1: exact consistency is achievable at rate 1 and impossible at any higher independent rate. We then prove a unified finite converse: a k-way ambiguity class requires at least $\log_2 k$ bits of zero-error side information, and the same finite budget yields conditional-entropy/Fano bounds for the latent source and decoder output, together with KL-based output-gap witnesses.

The result is a single deterministic finite theorem program with counting, confusability, entropy, decoder-output, and PMF/KL forms. Applications are corollaries; core theorem chains are machine-checked in Lean 4.

Index Terms: zero-error capacity, side information, multi-terminal source coding, exact consistency, confusability graph

I. Introduction

When a latent fact is represented at multiple locations and those representations may be modified over time, exact consistency becomes a zero-error information problem. An observer must either recover a unique authoritative value or face an ambiguity class of mutually incompatible states. This paper studies the information-theoretic structure of that problem.

Our starting point is an abstract multi-location encoding model. A fact F is represented across locations $\{L_1, \ldots, L_n\}$, some independent and some derived. We ask two questions:

1) What independent encoding rate guarantees exact consistency, i.e., zero probability of disagreement among reachable states?
2) When disagreement exists, how much side information is necessary to resolve the ambiguity without error?

This places the problem in the orbit of zero-error and multi-terminal information theory [1]–[4]. The relevant object is not ordinary vanishing-error capacity, but an exact-consistency analogue of zero-error resolvability under repeated updates.

T. Simas is with McGill University, Montreal, QC, Canada (e-mail: tristan.simas@mail.mcgill.ca).

Manuscript received March 6, 2026.

© 2026 Tristan Simas. This work is licensed under CC BY 4.0. License: https://creativecommons.org/licenses/by/4.0/

A. Zero-Incoherence Capacity

We define the **zero-incoherence capacity** C_0 as the largest independent encoding rate achieving exactly zero disagreement among reachable states. In our model, rate is the number of independent encoding locations (degrees of freedom, DOF). The central characterization is:

$$\boxed{C_0 = 1}$$

This is a zero-error style result in the Shannon tradition [1]–[3], adapted to modifiable storage/encoding systems rather than one-shot channels.

Main results. Let DOF (Degrees of Freedom) denote the encoding rate: the number of independent locations that can hold distinct values simultaneously. The paper has one sharp threshold theorem and one unified converse theorem family.

- **Zero-incoherence capacity (Theorems II.38–II.37):** rate 1 is achievable and any higher independent rate is impossible, so $C_0 = 1$ exactly (UNQ1).
- **Unified converse family (Theorem II.48 and Section VI):** once ambiguity survives the observation transcript, the same finite budget simultaneously limits zero-error side information, conditional entropy, decoder-output entropy, and output-side entropy/KL saturation; operationally, this yields $O(1)$ modification complexity at capacity and $\Omega(n)$ above it (Theorem VI.8).

Theorem I.1 (Unified Converse, informal)**.** *Let X be a finite latent source, let Y denote the deterministic observation/tag transcript, and let $\hat{X}$ be any deterministic decoder output. If the observer cannot separate all reachable alternatives exactly, then the same finite alphabet budget controls all of the following: exact zero-error resolution of ambiguity classes, the conditional entropy $H(X \mid Y)$, the decoder-output entropy $H(\hat{X})$, and the output-side entropy gaps $\log|\mathcal{Y}| - H(Y)$ and $\log|\hat{\mathcal{X}}| - H(\hat{X})$. In particular, exact k-way resolution requires at least $\log_2 k$ bits, while low error implies Fano-style upper bounds for both $H(X \mid Y)$ and $H(\hat{X})$.*

This is the paper's central synthesis: the exact-consistency problem is not only a zero-error capacity statement but a unified converse program connecting ambiguity classes, confusability structure, finite Fano inequalities, and KL-to-uniform saturation gaps.

B. Why This Is an Information-Theoretic Problem

The zero-incoherence threshold is stated in achievability/converse form:

Encoding Rate	Zero Incoherence?	Interpretation
DOF $= 0$	N/A	Fact not encoded
DOF $= 1$	**Yes**	Capacity-achieving
DOF > 1	No	Above capacity

Comparison to Shannon capacity. Shannon's channel capacity C is the supremum of rates R achieving vanishing error probability: $\lim_{n\to\infty} P_e^{(n)} = 0$. Our zero-incoherence capacity is the supremum of rates achieving *exactly zero* incoherence probability, so the closest analogy is to zero-error capacity [1], not ordinary capacity.

Operational meaning. The threshold identifies when ambiguity can be avoided entirely: one independent source can be propagated to many derived views without disagreement, while two or more independently writable sources create reachable ambiguity classes. The side-information theorem then quantifies the cost of resolving those classes once they exist.

C. Applications Across Domains

The abstract model applies wherever facts are stored redundantly:

- **Distributed databases:** Replica consistency under partition constraints [5]
- **Version control:** Merge resolution when branches diverge [6]
- **Configuration systems:** Multi-file settings with coherence requirements [7]
- **Software systems:** Class registries, type definitions, interface contracts [8]

In each domain the same question appears: when do replicated representations admit a unique zero-error interpretation, and when do they require external side information for resolution? The software examples later in the paper are applications, not the primary object.

D. Connection to Classical Information Theory

The paper sits at the intersection of three classical lines.

1. Multi-terminal source coding. Slepian-Wolf [4] characterizes distributed encoding of correlated sources. We model encoding locations as terminals: derivation introduces *perfect correlation* (deterministic dependence), reducing effective rate. The capacity result shows that only complete correlation (all terminals derived from one source) guarantees coherence–partial correlation permits divergence. Section II-L develops this connection.

2. Zero-error capacity and confusability. Shannon [1], Körner [2], and Lovász [3] characterize zero-error communication through distinguishability and confusability structure. We characterize **zero-incoherence encoding**–a storage analog where "errors" are disagreements among locations. The achievability/converse structure (Theorems II.38, II.39) is intentionally cast in that style, while the strengthened finite source model later exposes exact-recovery cliques, tag-alphabet bounds, and graph-style entropy consequences inside the same theorem stack (UNQ1, UNQ1).

3. Interactive information theory and finite Fano converses. Our model is interactive because encodings are modified over time, and causal propagation is analogous to feedback [9], [10]. The strengthened finite theorem chain then turns this storage/update model into a deterministic analogue of finite Fano reasoning: the same observation/tag budget controls $H(X \mid Y)$, $H(\hat{X})$, and the operational rate–complexity gap (Section VI).

E. Realizability and Applications

A secondary question is which concrete systems can realize the capacity-achieving regime ($C_0 = 1$). We prove realizability requires two information-theoretic properties:

1) **Causal update propagation (feedback coupling):** Changes to the source must automatically trigger updates to derived locations. This is analogous to *channel coding with feedback* [11]–the encoder (source) and decoder (derived locations) are coupled causally. Without feedback, a temporal window exists where source and derived locations diverge (temporary incoherence).
2) **Provenance observability (decoder side information):** The system must support queries about derivation structure. This is the encoding-system analog of *Slepian-Wolf side information* [4]–the decoder has access to structural information enabling verification that all terminals are derived from the source.

Theorem I.2 (Encoder Realizability, informal)**.** *An encoding system achieves $C_0 = 1$ iff it provides both causal propagation and provenance observability (UNQ1). Neither alone suffices (Theorem IV.8, UNQ1).*

Connection to multi-version coding. Rashmi et al. [12] prove an "inevitable storage cost for consistency" in distributed storage. Our realizability theorem is analogous: systems lacking either encoder property *cannot* achieve capacity–the constraint is information-theoretic, not implementation-specific.

Instantiations. The encoder properties instantiate across domains: programming languages, distributed databases, and configuration systems. Section V treats these only as corollaries of the abstract theory.

F. Paper Organization

Core theorems (capacity, realizability, complexity bounds) are machine-checked in Lean 4 [13] (14898 lines, 703 theorem/lemma statements, 0 `sorry` placeholders). The source proof tree now discharges the side-information/counting layer by finite zero-error arguments rather than paper-local assumption bundles.

Section II–Encoding Model and Capacity. We define multi-location encoding systems, encoding rate (DOF), and coherence/incoherence, then prove the exact-consistency threshold ($C_0 = 1$) with explicit achievability/converse structure and the base side-information lower bound.

Section III–Derivation and Optimal Rate. We characterize derivation as the mechanism achieving capacity: derived locations are perfectly correlated with their source, contributing zero effective rate.

Section IV–Encoder Realizability. We prove that realizing the threshold in concrete systems requires causal propagation (feedback) and provenance observability (decoder side information). Both are necessary; together sufficient.

Section VI–Unified Converse and Rate-Complexity Consequences. We develop the finite converse family in counting, clique, Fano-style, conditional-entropy, decoder-output, and PMF/KL forms, then connect it to the unbounded rate–complexity gap.

Sections V, VII–Applications (Corollaries). A language-level instantiation and a brief worked example. These illustrate the abstract theory; core results are domain-independent.

G. Scope

The abstract model covers facts represented at multiple locations under allowed edits. The concrete realizability results in this paper focus on *structural facts* (class existence, method signatures, type relationships), because these provide the clearest exact-consistency setting. The complexity analysis is asymptotic in the number of encoding locations.

Cross-system realizations. Multi-language or cross-runtime realizations typically require external code generation or synchronization layers. Those are natural applications of the model, but their detailed treatment is left for future work (Section IX).

H. Contributions

This paper makes three primary information-theoretic contributions and two supporting structural ones:

- **Exact-consistency threshold:** the model defines DOF, coherence, and ambiguity, and proves $C_0 = 1$ exactly by achievability and converse (Section II-K).
- **Unified converse family:** the side-information lower bound for k-way ambiguity classes is lifted into deterministic clique/confusability, conditional-entropy/Fano, decoder-output, and PMF/KL forms (Sections II-L and VI).
- **Encoder realizability:** causal propagation and provenance observability are necessary and sufficient to realize the capacity-achieving regime (Section IV).
- **Rate–complexity consequence:** the exact-consistency threshold induces an unbounded operational gap between the DOF $= 1$ and DOF > 1 regimes (Section VI).
- **Supporting structural analogies:** CAP/FLP-style statements appear as secondary consequences of the same abstract encoding model (Section II-I).

Applications (corollaries). Sections V and VII instantiate the realizability theorem in language-level terms and provide a worked example. These are illustrative corollaries; the core information-theoretic results are self-contained in Sections II–VI.

II. Encoding Systems and Coherence

We formalize exact consistency for multi-location encodings with modification constraints. The goal of this section is to isolate the abstract IT model before any software-specific interpretation: facts take values in finite alphabets, system states expose multiple representations of those facts, edits move the system among reachable states, and the observer must determine whether the represented fact is uniquely recoverable.

A. Model assumptions and notation

For rigour we state the modeling assumptions used throughout the paper. These are intentionally minimal and are made explicit so reviewers can judge applicability.

1) **Fact value model (A1):** Each fact F takes values in a finite set $\mathcal{V}_F$; $H(F) = \log_2 |\mathcal{V}_F|$ denotes its entropy under a chosen prior. When we write "zero probability of incoherence" we mean $\Pr(\text{incoherent}) = 0$ under the model for edits and randomness specified below.
2) **Update model (A2):** Modifications occur as discrete events (rounds). An edit δ_F changes the source value for F; derived locations update deterministically when causal propagation is present. We do not require a blocklength parameter; our asymptotic statements are in the number of encoding locations n or in ensemble scaling of the codebase.
3) **Adversary / randomness (A3):** When randomness is needed we assume a benign stochastic model for edits (e.g., uniformly sampled new values for illustration). For impossibility/converse statements we make no cooperative assumptions of the editor: adversarial sequences of edits that produce incoherence are allowed when DOF > 1.
4) **Side-information channel (A4):** Derived locations and runtime queries collectively form side information S available to any verifier. In the finite zero-error layer, the relevant quantity is the number of distinguishable tags that S can expose.

References to these assumptions use labels (A1)–(A4). Full formal versions of the operational model are given in the mechanized supplement (Supplement A).

B. The Encoding Model

We begin with the abstract encoding model: locations, values, reachable states, and ambiguity.

Definition II.1 (Encoding System)**.** An *encoding system* for a fact F is a collection of locations $\{L_1, \ldots, L_n\}$, each capable of holding a value for F.

Definition II.2 (Coherence)**.** An encoding system is *coherent* iff all locations hold the same value:

$$\forall i, j : \text{value}(L_i) = \text{value}(L_j)$$

Definition II.3 (Incoherence)**.** An encoding system is *incoherent* iff some locations disagree:

$$\exists i, j : \text{value}(L_i) \neq \text{value}(L_j)$$

Definition II.4 (Ambiguity Class)**.** For a system state x, the *ambiguity class* of x is the set

$$\mathcal{A}(x) = \{v : \exists L \text{ with value}(L) = v\}.$$

When $|\mathcal{A}(x)| > 1$, the state presents multiple internally supported candidate values for the same fact.

The zero-error resolution problem. Given a state x, a resolver must output the correct value of F with zero error. When $|\mathcal{A}(x)| > 1$, the internal state alone may fail to identify a unique answer.

Theorem II.5 (Oracle Arbitrariness)**.** *For any incoherent encoding system state x and any oracle O that resolves x to a value $v \in \mathcal{A}(x)$, there exists a value $v' \in \mathcal{A}(x)$ such that $v' \neq v$.*

(L: UNQ1, UNQ1, UNQ1)

Proof. By incoherence, $|\mathcal{A}(x)| > 1$, so there exist $v_1, v_2 \in \mathcal{A}(x)$ with $v_1 \neq v_2$. Either O picks v_1 (then v_2 disagrees) or it does not pick v_1 (then v_1 disagrees). ■

Interpretation. This is the first zero-error obstruction in the paper: once an ambiguity class contains more than one value, the state itself does not identify which value is authoritative. Any exact resolver must therefore use auxiliary information not contained in the state representation alone.

Definition II.6 (Degrees of Freedom)**.** The *degrees of freedom* (DOF) of an encoding system is the number of locations that can be modified independently.

Theorem II.7 (DOF = 1 Guarantees Coherence)**.** *If DOF = 1, then the encoding system is coherent in all reachable states.*

(L: UNQ1, UNQ1, UNQ1)

Proof. With DOF = 1, exactly one location is independent. All other locations are derived (automatically updated when the source changes). Derived locations cannot diverge from their source. Therefore, all locations hold the value determined by the single independent source. Disagreement is impossible. ■

Theorem II.8 (DOF $>$ 1 Permits Incoherence)**.** *If DOF $>$ 1, then incoherent states are reachable.*

(L: UNQ1, UNQ1, UNQ1)

Proof. With DOF $>$ 1, at least two locations are independent. Independent locations can be modified separately. A sequence of edits can set $L_1 = v_1$ and $L_2 = v_2$ where $v_1 \neq v_2$. This is an incoherent state. ■

Corollary II.9 (Coherence Forces DOF = 1)**.** *If coherence must be guaranteed (no incoherent states reachable), then DOF = 1 is necessary and sufficient.*

(L: UNQ1, UNQ1, UNQ1, UNQ1)

This is the basic exact-consistency criterion in the model (UNQ1, UNQ1, UNQ1).

The following subsections refine this model and then connect it back to concrete systems only as examples.

C. Computational Realizations

The abstract encoding model (Definitions II.1–II.6) applies to any system where:

1) Facts are encoded at multiple locations
2) Locations can be modified
3) Correctness requires coherence across modifications

Domains satisfying these constraints:

- **Software codebases:** Type definitions, registries, configurations
- **Distributed databases:** Replica consistency under updates
- **Configuration systems:** Multi-file settings (e.g., infrastructure-as-code)
- **Version control:** Merge resolution under concurrent modifications

We focus on *computational realizations* only to anchor the model. Software codebases are a convenient example, but the theorems in this section are stated independently of any language, runtime, or toolchain.

Definition II.10 (Codebase (Software Realization))**.** A *codebase C* is a finite collection of source files, each containing syntactic constructs (classes, functions, statements, expressions). This is the canonical computational encoding system.

Definition II.11 (Location)**.** A *location* $L \in C$ is a syntactically identifiable region: a class definition, function body, configuration value, type annotation, database field, or configuration entry.

Definition II.12 (Modification Space)**.** For encoding system C, the *modification space* $E(C)$ is the set of all valid modifications. Each edit $\delta \in E(C)$ transforms C into $C' = \delta(C)$.

The modification space is large (exponential in system size). But we focus on modifications that *change a specific fact*.

D. Facts: Atomic Units of Specification

Definition II.13 (Fact)**.** A *fact* F is an atomic unit of program specification: a single piece of knowledge that can be independently modified. Facts are the indivisible units of meaning in a specification.

The granularity of facts is determined by the specification, not the implementation. If two pieces of information must always change together, they constitute a single fact. If they can change independently, they are separate facts.

Examples of facts:

Fact	Description
F_1: "threshold = 0.5"	A configuration value
F_2: "`PNGLoader` handles `.png`"	A type-to-handler mapping
F_3: "`validate()` returns `bool`"	A method signature
F_4: "`Detector` is a subclass of `Processor`"	An inheritance relationship
F_5: "`Config` has field `name: str`"	A dataclass field

Definition II.14 (Structural Fact)**.** A fact F is *structural* with respect to encoding system C iff the locations encoding F are fixed at definition time:

$$\text{structural}(F, C) \iff \forall L : \text{encodes}(L, F) \rightarrow L \in \text{DefinitionSyntax}(C)$$

where $\text{DefinitionSyntax}(C)$ comprises declarative constructs that cannot change post-definition without recreation.

Examples across domains:

- **Software:** Class declarations, method signatures, inheritance clauses, attribute definitions
- **Databases:** Schema definitions, table structures, foreign key constraints
- **Configuration:** Infrastructure topology, service dependencies
- **Version control:** Branch structure, merge policies

Key property: Structural facts are fixed at *definition time*. Once defined, their structure cannot change without recreation. This is why structural coherence requires definition-time computation: the encoding locations are only mutable during creation.

Non-structural facts (runtime values, mutable state) have encoding locations modifiable post-definition. Achieving DOF = 1 for non-structural facts requires different mechanisms (reactive bindings, event systems) and is outside this paper's scope. We focus on structural facts because they demonstrate the impossibility results most clearly.

E. Encoding: The Correctness Relationship

Definition II.15 (Encodes)**.** Location L *encodes* fact F, written $\text{encodes}(L, F)$, iff correctness requires updating L when F changes.

Formally:

$$\text{encodes}(L, F) \iff \forall \delta_F : \neg\text{updated}(L, \delta_F) \rightarrow \text{incorrect}(\delta_F(C))$$

where δ_F is an edit targeting fact F.

Key insight: This definition is induced by correctness rather than by annotation. If failing to update location L when fact F changes produces an incorrect system, then L participates in the representation of F.

Example II.16 (Encoding in Practice)**.** Consider a type registry:

```
# Location L1: Class definition
class PNGLoader(ImageLoader):
    format = "png"

# Location L2: Registry entry
LOADERS = {"png": PNGLoader, "jpg": JPGLoader}

# Location L3: Documentation
# Supported formats: png, jpg
```

The fact F = "`PNGLoader` handles `png`" is encoded at:

- L_1: The class definition (primary encoding)
- L_2: The registry dictionary (secondary encoding)
- L_3: The documentation comment (tertiary encoding)

If F changes (e.g., to "`PNGLoader` handles `png` and `apng`"), all three locations must be updated for correctness. The program is incorrect if L_2 still says `{"png": PNGLoader}` when the class now handles both formats.

F. Modification Complexity

Definition II.17 (Modification Complexity)**.**

$$M(C, \delta_F) = |\{L \in C : \text{encodes}(L, F)\}|$$

The number of locations that must be updated when fact F changes.

Modification complexity is the central metric of this paper. It measures the *cost* of changing a fact. A codebase with $M(C, \delta_F) = 47$ requires 47 edits to correctly implement a change to fact F. A codebase with $M(C, \delta_F) = 1$ requires only 1 edit.

Theorem II.18 (Correctness Forcing)**.** *$M(C, \delta_F)$ is the **minimum** number of edits required for correctness. Fewer edits imply an incorrect program.*

(L: UNQ1)

Proof. Suppose $M(C, \delta_F) = k$, meaning k locations encode F. By Definition II.15, each encoding location must be updated when F changes. If only $j < k$ locations are updated, then $k - j$ locations still reflect the old value of F. These locations create inconsistencies:

1) The specification says F has value v' (new)
2) Locations $L_1, \ldots, L_j$ reflect v'
3) Locations $L_{j+1}, \ldots, L_k$ reflect v (old)

By Definition II.15, the program is incorrect. Therefore, all k locations must be updated, and k is the minimum. ■

G. Independence and Degrees of Freedom

Not all encoding locations are created equal. Some are *derived* from others.

Definition II.19 (Independent Locations)**.** Locations L_1, L_2 are *independent* for fact F iff they can diverge. Updating L_1 does not automatically update L_2, and vice versa.

Formally: L_1 and L_2 are independent iff there exists a sequence of edits that makes L_1 and L_2 encode different values for F.

Definition II.20 (Derived Location)**.** Location L_{derived} is *derived from* L_{source} iff updating L_{source} automatically updates L_{derived}. Derived locations are not independent of their sources.

Example II.21 (Independent vs. Derived)**.** Consider two architectures for the type registry:

Architecture A (independent locations):

```
# L1: Class definition
class PNGLoader(ImageLoader): ...

# L2: Manual registry (independent of L1)
LOADERS = {"png": PNGLoader}
```

Here L_1 and L_2 are independent. A developer can change L_1 without updating L_2, causing inconsistency.

Architecture B (derived location):

```
# L1: Class definition with registration
class PNGLoader(ImageLoader):
    format = "png"

# L2: Derived registry (computed from L1)
LOADERS = {cls.format: cls for cls in
    ImageLoader.__subclasses__()}
```

Here L_2 is derived from L_1. Updating the class definition automatically updates the registry. They cannot diverge.

Definition II.22 (Degrees of Freedom)**.**

$$\mathrm{DOF}(C, F) = |\{L \in C : \mathrm{encodes}(L, F) \wedge \mathrm{independent}(L)\}|$$

The number of *independent* locations encoding fact F.

DOF is the key metric. Modification complexity M counts all encoding locations. DOF counts only the independent ones. If all but one encoding location is derived, DOF = 1 even though M can be large.

Theorem II.23 (DOF = Incoherence Potential)**.** *$DOF(C, F) = k$ implies that up to k distinct values for F can coexist simultaneously. In particular, $k > 1$ makes ambiguity reachable.*

(L: UNQ1)

Proof. Each independent location can hold a different value. By Definition II.19, no constraint forces agreement between independent locations. Therefore, k independent locations can simultaneously realize up to k distinct values. ■

Corollary II.24 (DOF $>$ 1 Implies Incoherence Risk)**.** *$DOF(C, F) > 1$ implies incoherent states are reachable.*

(L: UNQ1)

H. The DOF Lattice

DOF values partition the model into three regimes:

DOF	Encoding Status
0	Fact F is not encoded (no representation)
1	Coherence guaranteed (optimal rate under coherence constraint)
$k > 1$	Incoherence possible (redundant independent encodings)

Theorem II.25 (DOF = 1 is Uniquely Coherent)**.** *For any fact F that must be encoded, $DOF(C, F) = 1$ is the unique value guaranteeing coherence:*

1) *DOF = 0: Fact is not represented*
2) *DOF = 1: Coherence guaranteed (by Theorem II.7)*
3) *DOF $>$ 1: Incoherence reachable (by Theorem II.8)*

This theorem is deliberately simple: it records the exact-consistency trichotomy of the model before the later side-information and complexity consequences are added. The same trichotomy is recast as a capacity statement in Section II-K.

(L: UNQ1)

I. Encoding-Theoretic CAP and FLP

We record two structural analogies to CAP and FLP inside the encoding model (UNQ1, UNQ1). These are secondary to the main IT contribution.

Definition II.26 (Local Availability)**.** An encoding system for fact F is *locally available* iff for every encoding location L of F and every value v, there exists a valid edit $\delta \in E(C)$ such that $\mathrm{updated}(L, \delta)$ and for every other encoding location L', $\neg\mathrm{updated}(L', \delta)$. Informally: each encoding location can be updated without coordinating with others.

Definition II.27 (Partition Tolerance)**.** An encoding system for fact F is *partition-tolerant* iff F is encoded at two or more locations:

$$|\{L \in C : \mathrm{encodes}(L, F)\}| \geq 2.$$

This is the minimal formal notion of "replication" in our model; without it, partitions are vacuous.

Theorem II.28 (CAP in the Encoding Model)**.** *No encoding system can simultaneously guarantee coherence (Definition II.2), local availability (Definition II.26), and partition tolerance (Definition II.27) for the same fact F.*

(L: UNQ1)

Proof. Partition tolerance gives at least two encoding locations. Local availability allows each to be updated without updating any other encoding location, so by Definition II.19 there exist two independent locations and thus $\mathrm{DOF}(C, F) > 1$. By Theorem II.8, incoherent states are reachable, contradicting coherence. ■

Definition II.29 (Resolution Procedure)**.** A *resolution procedure* is a deterministic function R that maps an encoding system state to a value present in that state.

Theorem II.30 (Static FLP in the Encoding Model)**.** *For any incoherent encoding system state and any resolution procedure R, the returned value is arbitrary relative to the other values present; no deterministic R can be justified by internal information alone.*

(L: UNQ1)

Proof. Immediate from Theorem II.5: in an incoherent state, at least two distinct values are present, and any choice leaves another value disagreeing. ■

These theorems are the encoding-theoretic counterparts of CAP [5], [14] and FLP [15]: CAP corresponds to the impossibility of coherence when replicated encodings remain independently updatable; FLP corresponds to the impossibility of truth-preserving resolution in an incoherent state without side information.

J. Information-Theoretic Quantities

Before establishing the main threshold result, we define the information-theoretic quantities used to express ambiguity, redundancy, and side-information requirements.

Definition II.31 (Encoding Rate)**.** The *encoding rate* of system C for fact F is $R(C, F) = \mathrm{DOF}(C, F)$, the number of independent encoding locations. This counts locations that can hold distinct values simultaneously.

Definition II.32 (Value Entropy)**.** For a fact F with value space $\mathcal{V}_F$, the *value entropy* is:

$$H(F) = \log_2 |\mathcal{V}_F|$$

This is the information content of specifying F's value.

Definition II.33 (Encoding Redundancy)**.** The *encoding redundancy* of system C for fact F is:

$$\rho(C, F) = \mathrm{DOF}(C, F) - 1$$

Redundancy measures excess independent encodings beyond the minimum needed to represent F.

Theorem II.34 (Redundancy-Incoherence Equivalence)**.** *Encoding redundancy $\rho > 0$ is necessary and sufficient for incoherence reachability:*

$$\rho(C, F) > 0 \iff \textit{incoherent states reachable}$$

(L: UNQ1)

Proof. ($\Rightarrow$) If $\rho > 0$, then DOF > 1. By Theorem II.8, incoherent states are reachable.

($\Leftarrow$) If incoherent states are reachable, then by contrapositive of Theorem II.7, DOF $\neq 1$. Since the fact is encoded, DOF ≥ 1, so DOF > 1, hence $\rho > 0$. ■

Definition II.35 (Incoherence Entropy)**.** For a state whose ambiguity class has size k, the *incoherence entropy* is:

$$H_{\text{inc}} = \log_2 k$$

This quantifies the uncertainty about which value is authoritative.

K. Zero-Incoherence Capacity Theorem

We now establish the central exact-consistency characterization. The argument is elementary at the combinatorial level, but it serves as the base layer for the later side-information and rate-complexity consequences. In that sense the section plays the role that an achievability/converse theorem plays in classical IT.

Background: Zero-Error Capacity. Shannon [1] introduced zero-error capacity: the maximum rate at which information can be transmitted with *zero* probability of error. Körner [2] and Lovász [3] linked such questions to confusability structure. Our zero-incoherence capacity is the corresponding exact-consistency threshold for modifiable encodings.

Definition II.36 (Zero-Incoherence Capacity)**.** The *zero-incoherence capacity* of an encoding system is:

$$C_0 = \sup\{R : \text{encoding rate } R \Rightarrow \text{incoherence probability} = 0\}$$

where "incoherence probability = 0" means no incoherent state is reachable under any modification sequence.

Theorem II.37 (Zero-Incoherence Capacity)**.** *The zero-incoherence capacity of any encoding system under independent modification is exactly 1:*

$$C_0 = 1$$

(L: UNQ1)

We state the result in achievability/converse form because later sections attach nontrivial side-information and complexity consequences to the same threshold.

Theorem II.38 (Achievability)**.** *Encoding rate DOF $= 1$ achieves zero incoherence. Therefore $C_0 \geq 1$.*

(L: UNQ1)

Proof. Let $\mathrm{DOF}(C, F) = 1$. By Definition II.22, exactly one location L_s is independent; all other locations $\{L_i\}$ are derived from L_s.

By Definition II.20, derived locations satisfy: $\text{update}(L_s) \Rightarrow \text{automatically_updated}(L_i)$. Therefore, for any reachable state:

$$\forall i : \text{value}(L_i) = f_i(\text{value}(L_s))$$

where f_i is the derivation function. All values are determined by L_s. Disagreement requires two locations with different determining sources, but only L_s exists. Therefore, no incoherent state is reachable. ■

Theorem II.39 (Converse)**.** *Encoding rate DOF > 1 does not achieve zero incoherence. Therefore $C_0 < R$ for all $R > 1$.*

(L: UNQ1)

Proof. Let $\mathrm{DOF}(C, F) = k > 1$. By Definition II.19, there exist locations L_1, L_2 that can be modified independently.

Construction of incoherent state: Consider the modification sequence:

1) δ_1: Set $L_1 = v_1$ for some $v_1 \in \mathcal{V}_F$
2) δ_2: Set $L_2 = v_2$ for some $v_2 \neq v_1$

Both modifications are valid (each location accepts values from $\mathcal{V}_F$). By independence, δ_2 does not affect L_1. The resulting state has:

$$\text{value}(L_1) = v_1 \neq v_2 = \text{value}(L_2)$$

By Definition II.3, this state is incoherent. Since it is reachable, zero incoherence is not achieved.

The converse is intentionally combinatorial: once two independently writable locations exist, one can construct a reachable disagreement state. Later sections refine the same obstruction using finite zero-error side-information counting arguments. ■

Proof of Theorem II.37. By Theorem II.38, $C_0 \geq 1$.

By Theorem II.39, $C_0 < R$ for all $R > 1$.

Therefore $C_0 = 1$ exactly. The bound is tight: achieved at DOF $= 1$, not achieved at any DOF > 1. ■

Comparison to Shannon capacity. The analogy to channel coding is structural rather than literal:

Aspect	Shannon (Channel)	This Work (Encoding)
Rate	Bits per channel use	Independent encoding locations
Constraint	Error probability $\to 0$	Incoherence probability $= 0$
Achievability	$R < C$ achieves	DOF $= 1$ achieves
Converse	$R > C$ fails	DOF > 1 fails
Capacity	$C = \max I(X;Y)$	$C_0 = 1$

The key difference is that Shannon capacity concerns communication over noisy channels, while the present threshold concerns exact consistency of stored representations under allowed edits.

Corollary II.40 (Capacity-Achieving Rate is Unique)**.** *DOF $= 1$ is the unique capacity-achieving encoding rate. No alternative achieves zero incoherence at higher rate.*

(L: L1)

Corollary II.41 (Redundancy Above Capacity)**.** *Any encoding with $\rho > 0$ (redundancy) operates above capacity and cannot guarantee coherence.*

(L: UNQ1)

1) Zero-Distortion Boundary: The model induces a degenerate zero-distortion boundary: the only rate achieving perfect coherence is rate 1.

Definition II.42 (Rate-Incoherence Function)**.** For $\epsilon \in [0, 1]$, define

$$R(\epsilon) = \inf\{R : \exists \text{ encoding achieving } P(\text{incoherence}) \leq \epsilon\}$$

where $\epsilon \in [0, 1]$ is the tolerable incoherence probability.

Theorem II.43 (Rate-Incoherence has a Sharp Zero-Error Threshold)**.** *At the zero-error point, the minimum achievable rate is exactly one:*

$$R(0) = 1.$$

For every $\epsilon > 0$, the model imposes no nontrivial upper bound on admissible independent rate.

(L: UNQ1)

Proof. The zero-error statement is exactly Theorem II.37. Once a positive incoherence budget is allowed, the model no longer forbids higher independent rate, so no analogous finite cutoff remains. ■

Interpretation. Unlike classical rate-distortion curves, the interesting point here is the sharp zero-error boundary: perfect coherence isolates a single admissible rate.

2) Design Necessity: The threshold result also yields a simple design corollary:

Theorem II.44 (Design Necessity)**.** *If an encoding system requires zero incoherence for correctness, then the encoding rate must satisfy DOF ≤ 1. This is a necessary condition; no design choice can circumvent it.*

(L: UNQ1)

Proof. Suppose a system requires $P(\text{incoherence}) = 0$ and has DOF > 1. By Theorem II.39, incoherent states are reachable. This contradicts the requirement. Therefore DOF ≤ 1 is necessary.

The bound is not an implementation artifact: with DOF > 1, independent locations can diverge by construction. ■

Corollary II.45 (Architectural Forcing)**.** *For any fact F requiring coherence guarantees, system architecture must either:*

1) *Achieve DOF$(F) = 1$ through derivation mechanisms, or*
2) *Accept that coherence cannot be guaranteed and implement resolution protocols with $\geq \log_2(\text{DOF})$ bits of side information.*

There is no third option.

(L: L1)

3) Coding Interpretation of Derivation: The coding-theoretic interpretation is compact: derivation is the capacity-achieving dependence structure. One location carries the independent value and the remaining locations are deterministic views. Those views improve observability without increasing independent rate.

4) Connection to Confusability Graphs: Körner [2] and Lovász [3] characterized zero-error capacity via *confusability graphs*: vertices are channel inputs, edges connect inputs that can produce the same output. Zero-error capacity is determined by the graph's independence number.

We can define an analogous *incoherence graph* for encoding systems:

Definition II.46 (Incoherence Graph)**.** For encoding system C and fact F, the incoherence graph $G = (V, E)$ has:

- Vertices $V =$ independent encoding locations
- Edge $(L_i, L_j) \in E$ iff L_i and L_j can hold different values (are truly independent)

Theorem II.47 (Incoherence Graph Characterization)**.** *Zero incoherence is achievable iff the incoherence graph has at most one vertex: $|V| \leq 1$.*

(L: UNQ1)

Proof. If $|V| = 0$, no locations encode F. If $|V| = 1$, DOF $= 1$ and zero incoherence is achieved by Theorem II.38. If $|V| \geq 2$, DOF ≥ 2 and incoherence is reachable by Theorem II.39. ■

This graph view is intentionally simple. Its value is not a deep graph invariant, but a compact bridge to zero-error language before the richer confusability bounds used later.

L. Side Information for Resolution

When an encoding system is incoherent, exact resolution requires external side information. This is the point where the paper becomes genuinely information-theoretic: ambiguity size induces a quantitative lower bound on the information a resolver must obtain.

1) The Multi-Terminal View: We can view an encoding system as a *multi-terminal source coding* problem [4], [11]:

- Each encoding location is a *terminal* that can transmit (store) a value
- The fact F is the *source* to be encoded

- *Derivation* introduces correlation: a derived terminal's output is a deterministic function of its source terminal
- The *decoder* (any observer querying the system) must reconstruct F's value

In Slepian-Wolf coding, correlated sources (X, Y) can be encoded at rates (R_X, R_Y) satisfying $R_X \geq H(X|Y)$, $R_Y \geq H(Y|X)$, $R_X + R_Y \geq H(X, Y)$. With perfect correlation ($Y = f(X)$), we have $H(X|Y) = 0$–the correlated source needs zero rate.

Derivation as perfect correlation. In our model, derivation creates perfect correlation: if L_d is derived from L_s, then value$(L_d) = f(\text{value}(L_s))$ deterministically. The "rate" of L_d is effectively zero–it contributes no independent information. This is why derived locations do not contribute to DOF.

Independence as zero correlation. Independent locations have no correlation. Each can hold any value in $\mathcal{V}_F$. The total "rate" is DOF $\cdot H(F)$, but with DOF > 1, the locations can encode *different* values–incoherence.

2) Side Information Bound:

Theorem II.48 (Side Information Requirement)**.** *Given a state whose ambiguity class has size k, resolving to the correct value requires at least $\log_2 k$ bits of side information.*

(L: UNQ1)

Proof. The ambiguity class presents k internally supported alternatives. By Theorem II.5, no internal rule distinguishes them.

Let $S \in \{1, \ldots, k\}$ denote the index of the correct source. The decoder must determine S to resolve correctly. Since S is uniformly distributed over k alternatives (by symmetry of the incoherent state):

$$H(S) = \log_2 k$$

Any resolution procedure is a function $R : \text{State} \to \mathcal{V}_F$. Without side information Y:

$$H(S|R(\text{State})) = H(S) = \log_2 k$$

because R can be computed from the state, which contains no information about which source is correct.

With side information Y that identifies the correct source:

$$H(S|Y) = 0 \Rightarrow I(S; Y) = H(S) = \log_2 k$$

Therefore, side information must expose at least $\log_2 k$ bits' worth of distinguishable tags. ■

Corollary II.49 (DOF = 1 Requires Zero Side Information)**.** *With DOF = 1, resolution requires 0 bits of side information.*

(L: UNQ1)

Proof. With $k = 1$, there is one independent location. $H(S) = \log_2 1 = 0$. No uncertainty exists about the authoritative source. ■

Corollary II.50 (Side Information Scales with Redundancy)**.** *The required side information equals $\log_2(1 + \rho)$ where ρ is encoding redundancy:*

$$\textit{Side information required} = \log_2(\textit{DOF}) = \log_2(1 + \rho)$$

(L: UNQ1)

Connection to Slepian-Wolf. In Slepian-Wolf coding, the decoder has side information Y and decodes X at rate $H(X|Y)$. Our setting inverts this: the decoder needs side information S (source identity) to "decode" (resolve) which value is correct. The required side information is exactly the entropy of the source-selection variable.

Connection to zero-error decoding. In zero-error channel coding, the decoder must identify the transmitted message with certainty. Without sufficient side information (channel structure), this is impossible. Similarly, without $\log_2 k$ bits identifying the authoritative source, zero-error resolution is impossible.

Example II.51 (Side Information in Practice)**.** Consider a configuration system with DOF = 3:

- `config.yaml`: `threshold: 0.5`
- `settings.json`: `"threshold": 0.7`
- `params.toml`: `threshold = 0.6`

Resolution requires $\log_2 3 \approx 1.58$ bits of side information:

- A priority ordering: "YAML > JSON > TOML" (encodes permutation → identifies source)
- A timestamp comparison: "most recent wins" (encodes ordering → identifies source)
- An explicit declaration: "params.toml is authoritative" (directly encodes source identity)

With DOF = 1, zero bits suffice–the single source is self-evidently authoritative.

3) Multi-Terminal Capacity Interpretation: The exact-consistency threshold (Theorem II.37) can be restated in multi-terminal language:

Corollary II.52 (Multi-Terminal Interpretation)**.** *An encoding system achieves zero incoherence iff all terminals are perfectly correlated (every terminal's output is a deterministic function of one source terminal). Equivalently: the correlation structure must be a tree with one root.*

(L: UNQ1, UNQ1)

Proof. Perfect correlation means DOF = 1 (only the root is independent). By Theorem II.38, this achieves zero incoherence. If any two terminals are independent (not perfectly correlated through the root), DOF ≥ 2, and by Theorem II.39, incoherence is reachable. ■

This connects to network information theory: in a multi-terminal network, achieving coherence requires complete dependence on a single source–no "multicast" of independent copies.

M. Structure Theorems: The Derivation Lattice

The set of derivation relations on an encoding system has algebraic structure. We characterize this structure and its computational implications (UNQ1, UNQ1, UNQ1).

Definition II.53 (Derivation Relation)**.** A *derivation relation* $D \subseteq L \times L$ on locations L is a directed relation where

$(L_s, L_d) \in D$ means L_d is derived from L_s. We require D be acyclic (no location derives from itself through any chain).

Definition II.54 (DOF under Derivation)**.** Given derivation relation D, the degrees of freedom is:

$$\text{DOF}(D) = |\{L : \nexists L'.(L', L) \in D\}|$$

The count of locations with no incoming derivation edges (source locations).

Theorem II.55 (Derivation Lattice)**.** *The set of derivation relations on a fixed set of locations L, ordered by inclusion, forms a bounded lattice:*

1) ***Bottom*** *($\bot$): $D = \emptyset$ (no derivations, DOF = $|L|$)*
2) ***Top*** *($\top$): Maximal acyclic D with DOF = 1 (all but one location derived)*
3) ***Meet*** *($\wedge$): $D_1 \wedge D_2 = D_1 \cap D_2$*
4) ***Join*** *($\vee$): $D_1 \vee D_2 =$ transitive closure of $D_1 \cup D_2$ (if acyclic)*

(L: UNQ1)

Proof. **Bottom:** $\emptyset$ is trivially a derivation relation with all locations independent.

Top: For n locations, a maximal acyclic relation has one source (root) and $n - 1$ derived locations forming a tree or DAG. DOF = 1.

Meet: Intersection of acyclic relations is acyclic. The intersection preserves only derivations present in both.

Join: If $D_1 \cup D_2$ is acyclic, its transitive closure is the smallest relation containing both. If cyclic, join is undefined (partial lattice).

Bounded: $\emptyset \subseteq D \subseteq \top$ for all valid D. ■

Theorem II.56 (DOF is Anti-Monotonic)**.** *DOF is anti-monotonic in the derivation lattice:*

$$D_1 \subseteq D_2 \Rightarrow DOF(D_1) \geq DOF(D_2)$$

More derivations imply fewer independent locations.

(L: UNQ1)

Proof. Adding a derivation edge (L_s, L_d) to D can only decrease DOF: if L_d was previously a source (no incoming edges), it now has an incoming edge and is no longer a source. Sources can only decrease or stay constant as derivations are added. ■

Corollary II.57 (Minimal DOF = 1 Derivations)**.** *A derivation relation D with DOF(D) = 1 is* minimal *iff removing any edge increases DOF.*

(L: UNQ1, UNQ1)

Computational implication: Given an encoding system, there can be multiple DOF-1-achieving derivation structures. The minimal ones use the fewest derivation edges–the most economical way to achieve coherence.

Representation model for complexity. For the algorithmic results below, we assume the derivation relation D is given explicitly as a DAG over the location set L. The input size is $|L| + |D|$, and all complexity bounds are measured in this explicit representation.

Theorem II.58 (DOF Computation Complexity)**.** *Given an encoding system with explicit derivation relation D:*

1) *Computing DOF(D) is $O(|L|+|D|)$ (linear in locations plus edges)*
2) *Deciding if DOF(D) = 1 is $O(|L| + |D|)$*
3) *Finding a minimal DOF-1 extension of D is $O(|L|^2)$ in the worst case*

(L: UNQ1)

Proof. **(1) DOF computation:** Count locations with in-degree 0 in the DAG. Single pass over edges: $O(|D|)$ to compute in-degrees, $O(|L|)$ to count zeros.

(2) DOF = 1 decision: Compute DOF, compare to 1. Same complexity.

(3) Minimal extension: Must connect $k-1$ source locations to reduce DOF from k to 1. Finding which connections preserve acyclicity requires reachability queries. Naive: $O(|L|^2)$. With better data structures (e.g., dynamic reachability): $O(|L| \cdot |D|)$ amortized. ■

III. Derivation as the Capacity-Achieving Structure

Having established the encoding model (Section II-B), we now identify the structural mechanism that attains the exact-consistency regime: derivation induces perfect dependence, so derived locations add visibility without adding independent rate.

A. Rate 1 as the Exact-Consistency Regime

Rate 1 is not merely a design preference. In the present model it is the exact-consistency threshold for facts encoded in systems with modification constraints (UNQ1, UNQ1, UNQ1).

Definition III.1 (Optimal Encoding (DOF = 1))**.** Encoding system C achieves *optimal encoding rate* for fact F iff:

$$\text{DOF}(C, F) = 1$$

Equivalently: exactly one independent encoding location exists for F. All other encodings are derived.

Interpretation. The language of "single authoritative source" is an application-level reading of a more basic IT statement: one independent source plus deterministic derivations is the unique zero-incoherence regime.

Equivalently, rate 1 means one independent encoding location and only deterministic views elsewhere.

Corollary III.2 (DOF = 1 Guarantees Determinacy)**.** *If DOF$(C, F) = 1$, then for all reachable states of C, the value of F is determinate: all encodings agree.* (This restates Theorem II.7 in terms of determinacy.)

(L: UNQ1, UNQ1, UNQ1)

As an application-level interpretation, this recovers Hunt & Thomas's "single, unambiguous, authoritative representation" [8].

Our contribution here is to show that the capacity-achieving structure is exactly the one in which all non-source locations

are deterministic functions of a single source (UNQ1, UNQ1, UNQ1).

Corollary III.3 (DOF = 1 Achieves $O(1)$ Update)**.** *If $DOF(C, F) = 1$, then coherence restoration requires $O(1)$ updates.* (See Theorem VI.2.)

(L: UNQ1)

Corollary III.4 (Uniqueness of Optimal Rate)**.** *DOF = 1 is the unique rate guaranteeing coherence.* (See Theorem II.37.)

(L: L1, UNQ1, UNQ1, UNQ1)

Corollary III.5 (Incoherence Under Redundancy)**.** *Multiple independent sources encoding the same fact permit incoherent states. $DOF > 1 \Rightarrow$ incoherence reachable.*

(L: UNQ1, UNQ1, UNQ1)

Proof. Direct application of Theorem II.8. With DOF > 1, independent locations can be modified separately, reaching states where they disagree. ■

See the formal Zero-Incoherence Capacity statement in Section II-K (Theorem II.37, UNQ1). The remainder of this section focuses on why derivation is the relevant dependence structure.

B. Information-theoretic proof sketches

We give concise proof sketches connecting the combinatorial DOF arguments to finite zero-error information-theoretic tools:

- **Counting converse:** To show that limited side information cannot enable exact recovery of F, apply the finite counting converse (Theorem VI.5, UNQ1). If K ambiguous states must be distinguished using an L-bit side-information tag, exact recovery requires $K \leq 2^L$, equivalently $L \geq \log_2 K$.
- **Confusability graph:** Lemma IV.1 converts combinatorial indistinguishability (cliques) into information lower bounds: in the finite zero-error model, a k-clique forces at least k tag values (UNQ1, UNQ1); globally, the required tag alphabet is lower-bounded by both worst-case fiber size (UNQ1) and the observation-alphabet occupancy floor (UNQ1), equivalently $|\mathcal{V}| \leq |\mathcal{O}| \cdot |\mathcal{T}|$ for any zero-error scheme (UNQ1). This gives a language-independent certificate that side information is insufficient in environments with limited introspection.
- **DOF implication:** If side information cannot supply enough distinguishable tags for exact recovery, the system must retain multiple independent encodings (higher DOF) to achieve correctness under modifications. Higher DOF incurs $\Omega(n)$ manual edits (Theorem VI.3, UNQ1), completing the converse chain from information constraints to modification complexity.

C. Rate-Complexity Tradeoff

The DOF metric creates a direct tradeoff between independent rate and manual synchronization cost.

Question: When fact F changes, how many manual updates are required to restore coherence?

- **DOF = 1:** $O(1)$ updates. The single source determines all derived locations automatically.
- **DOF = $n > 1$:** $\Omega(n)$ updates. Each independent location must be synchronized manually.

This is a rate-complexity consequence of the exact-consistency threshold: higher independent rate incurs higher synchronization cost, while DOF = 1 minimizes that cost.

Key insight: Many locations can encode F (high total encoding locations), but if DOF = 1, coherence restoration requires only 1 manual update. The derivation mechanism handles propagation automatically.

Example III.6 (Encoding Rate vs. Modification Complexity)**.** Consider an encoding system where a fact F = "all processors must implement operation P" is encoded at 51 locations:

- 1 abstract specification location
- 50 concrete implementation locations

Architecture A (DOF = 51): All 51 locations are independent.

- Modification complexity: Changing F requires 51 manual updates
- Coherence risk: After $k < 51$ updates, system is incoherent (partial updates)
- Only after all 51 updates is coherence restored

Architecture B (DOF = 1): The abstract specification is the single source; implementations are derived.

- Modification complexity: Changing F requires 1 update (the specification)
- Coherence guarantee: Derived locations update automatically via enforcement mechanism
- The *specification* has a single authoritative source

Realization: Abstract base classes with enforcement (type checkers, runtime validation) achieve DOF = 1 for contract specifications. Changing the abstract method signature updates the contract; type checkers flag non-compliant implementations.

Note: Implementations are separate facts. DOF = 1 for the contract specification does not eliminate implementation updates–it ensures the specification itself is determinate.

D. Derivation: The Coherence Mechanism

Derivation is the mechanism by which DOF is reduced without losing encodings. A derived location cannot diverge from its source, eliminating it as a source of incoherence.

Definition III.7 (Derivation)**.** Location L_{derived} is *derived from* L_{source} for fact F iff:

$$\text{updated}(L_{\text{source}}) \rightarrow \text{automatically_updated}(L_{\text{derived}})$$

No manual intervention is required. Coherence is maintained automatically.

Derivation can occur at different times depending on the encoding system:

Derivation Time	Examples Across Domains
Compile/Build time	C++ templates, Rust macros, database schema generation, infrastructure-as-code compilation
Definition time	Python metaclasses, ORM model registration, dynamic schema creation
Query/Access time	Database views, computed columns, lazy evaluation

Structural facts require definition-time derivation. Structural facts (class existence, schema structure, service topology) are fixed when defined. Compile-time derivation that runs before the definition is fixed is too early (the declarative source is not yet fixed). Runtime is too late (structure already immutable). Definition-time is the unique opportunity for structural derivation—a consequence of timing trichotomy (UNQ1, UNQ1, UNQ1, UNQ1).

Theorem III.8 (Derivation Preserves Coherence)**.** *If $L_{derived}$ is derived from L_{source}, then $L_{derived}$ cannot diverge from L_{source} and does not contribute to DOF.*

(L: UNQ1, UNQ1, UNQ1)

Proof. By Definition III.7, derived locations are automatically updated when the source changes. Let L_d be derived from L_s. If L_s encodes value v, then L_d encodes $f(v)$ for some function f. When L_s changes to v', L_d automatically changes to $f(v')$.

There is no reachable state where $L_s = v'$ and $L_d = f(v)$ with $v' \neq v$. Divergence is impossible. Therefore, L_d does not contribute to DOF. ■

Corollary III.9 (Derivation Achieves Coherence)**.** *If all encodings of F except one are derived from that one, then $DOF(C, F) = 1$ and coherence is guaranteed.*

(L: UNQ1, UNQ1)

Proof. Let L_s be the non-derived encoding. All other encodings $L_1, \ldots, L_k$ are derived from L_s. By Theorem III.8, none can diverge. Only L_s is independent. Therefore, $\text{DOF}(C, F) = 1$, and by Theorem II.7, coherence is guaranteed. ■

E. Illustrative Realizations

This subsection is illustrative only. It shows how the dependence structure appears in concrete systems, but the theorem itself is independent of these realizations.

Software: Subclass Registration (Python)

```
class Registry:
    _registry = {}
    def __init_subclass__(cls, **kwargs):
        Registry._registry[cls.__name__] = cls

class PNGHandler(Registry):  # Automatically
    registered
    pass
```

Encoding structure:

- Source: Class definition (declared once)
- Derived: Registry dictionary entry (computed at definition time via `__init_subclass__`)
- DOF = 1: Registry cannot diverge from class hierarchy

Databases: Materialized Views

```
CREATE TABLE users (id INT, name TEXT, created_at
    TIMESTAMP);
CREATE MATERIALIZED VIEW user_count AS
  SELECT COUNT(*) FROM users;
```

Encoding structure:

- Source: Base table `users`
- Derived: Materialized view `user_count` (updated on refresh)
- DOF = 1: View cannot diverge from base table (consistency guaranteed by DBMS)

Configuration: Infrastructure as Code (Terraform)

```
resource "aws_instance" "app" {
  ami = "ami-12345"
  instance_type = "t2.micro"
}

output "instance_ip" {
  value = aws_instance.app.public_ip
}
```

Encoding structure:

- Source: Resource declaration (authoritative configuration)
- Derived: Output value (computed from resource state)
- DOF = 1: Output cannot diverge from actual resource (computed at apply time)

Common pattern: In all cases, one location carries the independent value and the others are deterministic views computed at definition/build/query time. Manual synchronization disappears because the dependence structure, not human process, enforces consistency.

IV. Realizability Requirements for the Capacity-Achieving Regime

This section identifies, at an abstract information-theoretic level, the two properties an encoding system must provide to realize the capacity-achieving regime. Concrete system-specific instantiations are treated as applications; here we isolate the abstract realizability conditions and their necessity/sufficiency proofs.

Informally, an encoding system provides a set of locations holding values and a space of allowed modifications. DOF = 1 is realizable when exactly one independent location exists (the source) and all other encodings are deterministic functions (derivations) of that source so divergence is impossible.

We show that two encoder properties are necessary and sufficient (UNQ1):

1) **Causal update propagation:** updates to the source must automatically propagate to derived locations (eliminating transient incoherence windows);
2) **Provenance observability:** the system must expose, to verification procedures, which locations are sources and which are derived (providing the side information necessary to verify DOF = 1).

The remainder of this section gives formal statements and proofs of necessity and sufficiency for these properties in the abstract model.

A. Confusability and Side-Information

To connect realizability to the information available at runtime, define the *confusability graph* for a fact F: vertices are distinct values of F, and an undirected edge joins two values x, x' iff they are indistinguishable given the available side information (i.e., no observation of derived locations can reliably separate x from x').

Lemma IV.1 (Confusability Clique Bound)**.** *If the confusability graph for F contains a clique of size k, then any zero-error side-information scheme for distinguishing all k values requires at least k distinct tags, equivalently at least $\log_2 k$ bits of side information.*

(L: UNQ1, UNQ1, UNQ1, UNQ1, UNQ1)

Proof sketch. Restrict F to the k values in the clique. In the finite zero-error setting, any zero-error tag assignment is a proper coloring of the confusability graph, so a k-clique forces at least k distinct tag values (UNQ1, UNQ1). More generally, the Lean observer/tag layer now proves three stronger structural facts: exact recovery is equivalent to injectivity of the combined observation-plus-tag map (OBS1, OBS2); each observation fiber requires at most as many states as available tags (OBS3); and every clique therefore satisfies the direct alphabet bound $|\mathcal{C}| \leq |\mathcal{T}|$ (OBS5). Consequently, the required tag alphabet is bounded below by the worst-case observation-fiber size, and globally every zero-error scheme must satisfy $|\mathcal{V}| \leq |\mathcal{O}| \cdot |\mathcal{T}|$ (OBS4). Theorem VI.5 packages the same obstruction as a finite counting converse: if only 2^L tags are available, then no zero-error resolver can distinguish more than 2^L ambiguous states. ■

The confusability graph now has a fully mechanized finite zero-error interpretation: large cliques force large tag alphabets (OBS5), nontrivial cliques are impossible for observation-only architectures (OBS6), the worst-case fiber bound captures the same obstruction globally (OBS3), and a global counting converse gives $|\mathcal{V}| \leq |\mathcal{O}| \cdot |\mathcal{T}|$ for any zero-error scheme (OBS4).

B. The Structural Timing Constraint

For certain classes of facts–*structural facts*–there is a fundamental timing constraint that shapes realizability.

Definition IV.2 (Structural Fact)**.** A fact F is *structural* if its encoding locations are fixed at the moment of definition. After definition, the structure cannot be retroactively modified–only new structures can be created.

Examples across domains:

- **Programming languages**: Class definitions, method signatures, inheritance relationships
- **Databases**: Schema definitions, table structures, foreign key constraints
- **Configuration systems**: Resource declarations, dependency specifications
- **Version control**: Branch structures, commit ancestry

The key property: structural facts have a *definition moment* after which their encoding is immutable. This creates a timing constraint for derivation.

Theorem IV.3 (Timing Constraint for Structural Derivation)**.** *For structural facts, derivation must occur at or before the moment the structure is fixed.*

(L: UNQ1)

Proof. Let F be a structural fact. Let t_{fix} be the moment F's encoding is fixed. Any derivation D that depends on F must execute at some time t_D.

Case 1: $t_D < t_{\text{fix}}$. Derivation executes before F is fixed. D cannot derive from F because F does not yet exist.

Case 2: $t_D > t_{\text{fix}}$. Derivation executes after F is fixed. D can read F but cannot modify structures derived from F–they are already fixed.

Case 3: $t_D = t_{\text{fix}}$. Derivation executes at the moment F is fixed. D can both read F and create derived structures before they are fixed.

Therefore, structural derivation requires $t_D = t_{\text{fix}}$. ■

This timing constraint is the information-theoretic reason why derivation must be *causal*–triggered by the act of defining the source, not by later access (UNQ1, UNQ1).

C. Requirement 1: Causal Update Propagation

Definition IV.4 (Causal Update Propagation)**.** An encoding system has *causal update propagation* if changes to a source location automatically trigger updates to all derived locations, without requiring explicit synchronization commands.

Formally: let L_s be a source location and L_d a derived location. The system has causal propagation iff:

$$\text{update}(L_s, v) \Rightarrow \text{automatically_updated}(L_d, f(v))$$

where f is the derivation function. No separate "propagate" or "sync" operation is required.

Information-theoretic interpretation: Causal propagation is analogous to *channel coding with feedback*. In classical channel coding, the encoder sends a message and waits for acknowledgment. With feedback, the encoder can immediately react to channel state. Causal propagation provides "feedback" from the definition event to the derivation mechanism–the encoder (source) and decoder (derived locations) are coupled in real-time.

Connection to multi-version coding: Rashmi et al. [12] formalize consistent distributed storage where updates to a source must propagate to replicas while maintaining consistency. Their "multi-version code" requires that any c servers can decode the latest common version–a consistency guarantee analogous to our coherence requirement. Causal propagation is the mechanism by which this consistency is maintained under updates.

Why causal propagation is necessary:

Without causal propagation, there exists a temporal window between source modification and derived location update. During this window, the system is incoherent–the source and derived locations encode different values.

Theorem IV.5 (Causal Propagation is Necessary for DOF = 1)**.** *Achieving DOF = 1 for structural facts requires causal update propagation.*

(L: UNQ1)

Proof. By Theorem IV.3, structural derivation must occur at definition time. Without causal propagation, derived locations are not updated when the source is defined. This means:

1) The source exists with value v
2) Derived locations have not been updated; they either do not exist yet or hold stale values
3) The system is temporarily incoherent

For DOF = 1, incoherence must be *impossible*, not merely transient. Causal propagation eliminates the temporal window: derived locations are updated *as part of* the source definition, not after.

Contrapositive: If an encoding system lacks causal propagation, DOF = 1 for structural facts is unrealizable. ■

Realizations across domains:

Domain	Causal Propagation Mechanism
Python	`__init_subclass__`, metaclass `__new__`
CLOS	`:after` methods on class initialization
Smalltalk	Class creation protocol, `subclass:` method
Databases	Triggers on schema operations (PostgreSQL event triggers)
Distributed systems	Consensus protocols (Paxos, Raft)
Configuration	Terraform dependency graph, reactive bindings

Systems lacking causal propagation:

- **Java**: Annotations are metadata, not executable. No code runs at class definition.
- **C++**: Templates expand at compile time but don't execute arbitrary user code.
- **Go**: No hook mechanism. Interface satisfaction is implicit.
- **Rust**: Proc macros run at compile time but generate static code, not runtime derivation.

D. Requirement 2: Provenance Observability

Definition IV.6 (Provenance Observability)**.** An encoding system has *provenance observability* if the system supports queries about derivation structure:

1) What locations exist encoding a given fact?
2) Which locations are sources vs. derived?
3) What is the derivation relationship (which derived from which)?

Information-theoretic interpretation: Provenance observability is the encoding-system analog of *side information at the decoder*. In Slepian-Wolf coding [4], the decoder has access to correlated side information that enables decoding at rates below the source entropy. Provenance observability provides "side information" about the encoding structure itself–enabling verification that DOF = 1 holds.

Without provenance observability, the encoding system is a "black box"–you can read locations but cannot determine which are sources and which are derived. This makes DOF uncomputable from within the system.

Theorem IV.7 (Provenance Observability is Necessary for Verifiable DOF = 1)**.** *Verifying that DOF = 1 holds requires provenance observability.*

(L: UNQ1)

Proof. Verification of DOF = 1 requires confirming:

1) All locations encoding fact F are enumerable
2) Exactly one location is independent (the source)
3) All other locations are derived from that source

Step (1) requires querying what structures exist. Step (2) requires distinguishing sources from derived locations. Step (3) requires querying the derivation relationship.

Without provenance observability, none of these queries are answerable from within the system. DOF = 1 can hold but cannot be verified. Bugs in derivation logic go undetected until coherence violations manifest. ■

Connection to coding theory: In coding theory, a code's structure (generator matrix, parity-check matrix) must be known to the decoder. Provenance observability is analogous: the derivation structure must be queryable for verification.

Realizations across domains:

Domain	Provenance Observability Mechanism
Python	`__subclasses__()`, `__mro__`, `dir()`, `vars()`
CLOS	`class-direct-subclasses`, MOP introspection
Smalltalk	`subclasses`, `allSubclasses`
Databases	System catalogs (`pg_depend`), query plan introspection
Distributed systems	Vector clocks, provenance tracking, `etcd` watch
Configuration	Terraform `graph`, Kubernetes API server

Systems lacking provenance observability:

- **C++**: Cannot query "what types instantiated template `Foo<T>`?"
- **Rust**: Proc macro expansion is opaque at runtime.
- **TypeScript**: Types are erased. Runtime cannot query type relationships.
- **Go**: No type registry. Cannot enumerate interface implementations.

E. Independence of Requirements

The two requirements–causal propagation and provenance observability–are independent. Neither implies the other (UNQ1, UNQ1). This section is included because it explains when the abstract IT optimum can actually be instantiated; it is not needed to state the main threshold theorem itself.

Theorem IV.8 (Requirements are Independent)**.** 1) *An encoding system can have causal propagation without provenance observability*
2) *An encoding system can have provenance observability without causal propagation*

(L: UNQ1)

Proof. **(1) Causal without provenance:** Rust proc macros execute at compile time (causal propagation: definition triggers code generation). But the generated code is opaque at runtime–the program cannot query what was generated (no provenance observability).

(2) Provenance without causal: Java provides reflection (`Class.getMethods()`, `Class.getInterfaces()`)–provenance observability. But no code executes when a class is defined–no causal propagation. ■

This independence means both requirements must be satisfied for DOF = 1 realizability (UNQ1).

F. The Realizability Theorem

Theorem IV.9 (Necessary and Sufficient Realizability Conditions)**.** *An encoding system S can achieve verifiable DOF = 1 for structural facts if and only if:*

1) *S provides causal update propagation, AND*
2) *S provides provenance observability*

(L: UNQ1)

Proof. ($\Rightarrow$) **Necessity:** Suppose S achieves verifiable DOF = 1 for structural facts.

- By Theorem IV.5, S must provide causal propagation
- By Theorem IV.7, S must provide provenance observability

($\Leftarrow$) **Sufficiency:** Suppose S provides both capabilities.

- Causal propagation enables derivation at the right moment (when structure is fixed)
- Provenance observability enables verification that all secondary encodings are derived
- Therefore, DOF = 1 is achievable: create one source, derive all others causally, verify completeness via provenance queries

■

Definition IV.10 (DOF-1-Complete Encoding System)**.** An encoding system is *DOF-1-complete* if it satisfies both causal propagation and provenance observability. Otherwise it is *DOF-1-incomplete*.

Information-theoretic interpretation: DOF-1-completeness is analogous to *channel capacity achievability*. A channel achieves capacity if there exist codes that approach the Shannon limit. An encoding system is DOF-1-complete if there exist derivation mechanisms that achieve the coherence-optimal rate (DOF = 1). The two requirements (causal propagation, provenance observability) are the "channel properties" that enable capacity achievement.

G. Connection to Write-Once Memory Codes

Our realizability requirements connect to *write-once memory (WOM) codes* [16], [17], an established area of coding theory.

A WOM is a storage medium where bits can only transition in one direction (typically $0 \to 1$). Rivest and Shamir [16] showed that WOMs can store more information than their apparent capacity by encoding multiple "writes" cleverly–the capacity for t writes is $\log_2(t+1)$ bits per cell.

The connection to our framework:

- **WOM constraint**: Bits can only increase (irreversible state change)
- **Structural fact constraint**: Structure is fixed at definition (irreversible encoding)
- **WOM coding**: Clever encoding enables multiple logical writes despite physical constraints
- **DOF = 1 derivation**: Clever derivation enables multiple logical locations from one physical source

Both settings involve achieving optimal encoding under irreversibility constraints. WOM codes achieve capacity via coding schemes; DOF-1-complete systems achieve coherence via derivation mechanisms.

H. The Logical Chain (Summary)

Observation: Structural facts are fixed at definition time (irreversible encoding).
$\downarrow$ (timing analysis)
Theorem IV.3: Derivation for structural facts must occur at definition time.
$\downarrow$ (requirement derivation)
Theorem IV.5: Causal update propagation is necessary for DOF = 1.
Theorem IV.7: Provenance observability is necessary for verifiable DOF = 1.
$\downarrow$ (conjunction)
Theorem IV.9: An encoding system achieves DOF = 1 iff it has both properties.
$\downarrow$ (evaluation)
Classification: Python, CLOS, Smalltalk are DOF-1-complete. Java, C++, Rust, Go are DOF-1-incomplete.

Mechanization status. The core requirement chain is machine-checked in Lean 4; current build statistics are 14898 lines, 703 theorem/lemma statements, and 0 `sorry` placeholders.

I. Concrete Impossibility Demonstration

We demonstrate exactly why DOF-1-incomplete systems cannot achieve DOF = 1 for structural facts.

The structural fact: "`PNGHandler` handles `.png` files." This fact must be encoded in two places:

1) The handler definition (where the handler is defined)
2) The registry/dispatcher (where format→handler mapping lives)

Python (DOF-1-complete) achieves DOF = 1:

```
class ImageHandler:
    _registry = {}

    def __init_subclass__(cls, format=None,
    **kwargs):
        super().__init_subclass__(**kwargs)
        if format:
            ImageHandler._registry[format] = cls
    # DERIVED (causal)

class PNGHandler(ImageHandler, format="png"):  #
    SOURCE
    def load(self, path): ...
```

Causal propagation: When `class PNGHandler` executes, `__init_subclass__` fires immediately, adding the registry entry. No temporal gap.

Provenance observability: `ImageHandler.__subclasses__()` returns all handlers. The derivation structure is queryable.

DOF = 1: The `format="png"` in the class definition is the single source. The registry entry is derived causally. Adding a new handler requires changing exactly one location.

Java (DOF-1-incomplete) cannot achieve DOF = 1:

```
// File 1: PNGHandler.java
@Handler(format = "png")  // Metadata, not
    executable
public class PNGHandler implements ImageHandler {
    ... }

// File 2: HandlerRegistry.java (SEPARATE SOURCE)
public class HandlerRegistry {
    static { register("png", PNGHandler.class); }
    // Manual
}
```

No causal propagation: The `@Handler` annotation is data, not code. Nothing executes when the class is defined.

Provenance partially present: Java has reflection, but cannot enumerate "all classes with `@Handler`" without classpath scanning.

DOF = 2: The annotation and the registry are independent encodings. Either can be modified without the other. Incoherence is reachable.

Theorem IV.11 (Generated Files Are Independent Sources)**.** *A generated source file constitutes an independent encoding, not a derivation. Code generation does not achieve DOF = 1.*

(L: UNQ1)

Proof. Let E_1 be the annotation on `PNGHandler.java`. Let E_2 be the generated `HandlerRegistry.java`.

Test: If E_2 is deleted or modified, does system behavior change? **Yes**–the handler is not registered.

Test: Can E_2 diverge from E_1? **Yes**–E_2 is a separate file that can be edited, fail to generate, or be stale.

Therefore, E_1 and E_2 are independent encodings. The fact that E_2 was *generated from* E_1 does not make it derived in the DOF sense, because:

1) E_2 exists as a separate artifact that can diverge
2) The generation process is external to the runtime and can be bypassed
3) There is no causal coupling–modification of E_1 does not automatically update E_2

Contrast with Python: the registry entry exists only in memory, created causally by the class statement. There is no second file. DOF = 1. ■

J. Summary: The Information-Theoretic Requirements

Requirement	IT Interpretation	Why Necessary
Causal propagation	Channel with feedback; encoder-decoder coupling	Eliminates temporal incoherence window
Provenance observability	Side information at decoder; codebook visibility	Enables DOF verification

Both requirements are necessary. Neither is sufficient alone. Together they enable DOF-1-complete encoding systems that achieve the coherence-optimal rate.

V. Application: Programming-Language Instantiation

We instantiate Theorem IV.9 in the domain of programming languages. This section is intentionally application-level: the abstract IT results are complete without it, and the goal here is only to show how the realizability theorem specializes once a language's definition-time hooks and introspection capabilities are fixed.

Corollary V.1 (Language Realizability Criterion)**.** *A programming language can realize DOF = 1 for structural facts iff it provides both (i) definition-time hooks and (ii) introspectable derivations. This is the direct instantiation of Theorem IV.9.*

(L: UNQ1)

Instantiation map. In the abstract model, an independent encoding is a location that can diverge under edits. In programming languages, structural facts are encoded at definition sites; *definition-time hooks* implement derivation (automatic propagation), and *introspection* implements provenance observability. Thus DEF corresponds to causal propagation and INTRO corresponds to queryable derivations; DOF = 1 is achievable exactly when both are present.

We instantiate this corollary over a representative language class (Definition V.2).

Definition V.2 (Mainstream Language)**.** A language is *mainstream* iff it appears in the top 20 of at least two of the following indices consistently over 5+ years:

1) TIOBE Index [18] (monthly language popularity)
2) Stack Overflow Developer Survey (annual)
3) GitHub Octoverse (annual repository statistics)
4) RedMonk Programming Language Rankings (quarterly)

A. Evaluation Criteria

The abstract realizability conditions (causal update propagation and provenance observability) map directly to concrete language capabilities (definition-time hooks and runtime introspection). We summarize this mapping briefly and defer the full language-by-language evaluation, extended code examples, and detailed rationale to Supplement A.

B. Summary Classification

Theorem IV.9 partitions languages into two classes:

Language	Def-time hooks	Introspection
DOF-1-complete (both requirements):		
Python	UNQ1	UNQ1
CLOS	`:after` methods	MOP queries
Smalltalk	class creation protocol	`subclasses`
DOF-1-incomplete (missing ≥ 1 requirement):		
Java	–	reflection
C++	–	–
Rust	proc macros (compile-time)	–
Go	–	–

Interpretation. DOF-1-complete languages can realize the abstract capacity-achieving structure for structural facts using language-native mechanisms. DOF-1-incomplete languages require external tooling (code generation, IDE refactoring) which operates outside language semantics and can be bypassed (cf. Theorem IV.8 on why external tools do not establish derivation).

Viewed after formalization, the two requirements may appear straightforward. In this mainstream set, only Python satisfies both; Java has introspection without definition-time hooks, Rust has compile-time hooks without runtime introspection, and C++ and Go have neither.

Supplement A contains the complete evaluation with justifications, worked code examples demonstrating each mechanism, and discussion of edge cases.

VI. Rate-Complexity Bounds

This section translates the exact-consistency threshold into an operational consequence: independent rate governs manual synchronization cost. The key point is not merely that rate 1 is coherent, but that moving above that threshold creates a provable complexity gap that grows with system size (UNQ1).

A. Cost Model

Definition VI.1 (Modification Cost Model)**.** Let δ_F be a modification to fact F in encoding system C. The *effective modification complexity* $M_{\text{effective}}(C, \delta_F)$ is the number of syntactically distinct edit operations that must be performed manually. Formally:

$$M_{\text{effective}}(C, \delta_F) = |\{L \in \text{Locations}(C) : \text{requires_manual_edit}(L, \delta_F)\}|$$

where $\text{requires_manual_edit}(L, \delta_F)$ holds iff location L must be updated manually (not by automatic derivation) to maintain coherence after δ_F.

Unit of cost: One edit = one syntactic modification to one location. We count locations, not keystrokes or characters. This abstracts over edit complexity to focus on the scaling behavior.

What we measure: Manual edits only. Derived locations that update automatically have zero cost. This isolates the operational consequence of independent information: if a location carries its own freely editable value, it contributes to synchronization burden.

Asymptotic parameter: We measure scaling in the number of encoding locations for fact F. Let $n = |\{L \in C : \text{encodes}(L, F)\}|$ and $k = \text{DOF}(C, F)$. Bounds of $O(1)$ and $\Omega(n)$ are in this parameter; in particular, the lower bound uses $n = k$ independent locations.

B. Upper Bound: DOF = 1 Achieves O(1)

Theorem VI.2 (DOF = 1 Upper Bound)**.** *For an encoding system with DOF = 1 for fact F:*

$$M_{\textit{effective}}(C, \delta_F) = O(1)$$

Effective modification complexity is constant regardless of system size.

(L: UNQ1)

Proof. Let $\text{DOF}(C, F) = 1$. By Definition III.1, C has exactly one independent encoding location. Let L_s be this single independent location.

When F changes:

1) Update L_s (1 manual edit)
2) All derived locations $L_1, \ldots, L_k$ are automatically updated by the derivation mechanism
3) Total manual edits: 1

The number of derived locations k can grow with system size, but the number of *manual* edits remains 1. Therefore, $M_{\text{effective}}(C, \delta_F) = O(1)$. ■

Note on "effective" vs. "total" complexity: Total modification complexity $M(C, \delta_F)$ counts all locations that change. Effective modification complexity counts only manual edits. With DOF = 1, total complexity can be $O(n)$ (many derived locations change), but effective complexity is $O(1)$ (one manual edit).

C. Lower Bound: DOF > 1 Requires $\Omega(n)$

Theorem VI.3 (DOF > 1 Lower Bound)**.** *For an encoding system with DOF > 1 for fact F, if F is encoded at n independent locations:*

$$M_{\textit{effective}}(C, \delta_F) = \Omega(n)$$

(L: UNQ1)

Proof. Let $\text{DOF}(C, F) = n$ where $n > 1$.

By Definition II.19, the n encoding locations are independent–updating one does not automatically update the others. When F changes:

1) Each of the n independent locations must be updated manually

2) No automatic propagation exists between independent locations
3) Total manual edits: n

Therefore, $M_{\text{effective}}(C, \delta_F) = \Omega(n)$. ■

D. Information-theoretic converse (finite counting)

To make the converse direction genuinely information-theoretic, we give a finite zero-error counting converse showing that insufficient side-information alphabet size makes exact recovery impossible (UNQ1, UNQ1). This is the bridge from ambiguity to complexity: if side information is insufficient, exact resolution fails, and systems that still demand correctness must pay through additional independent encodings and manual synchronization.

Remark VI.4 (Formalization status). The results in this subsection are formalized in the source Lean tree as finite zero-error counting theorems rather than probabilistic Fano inequalities. The core capacity, side-information, realizability, and complexity chains are machine-checked in that exact form.

Theorem VI.5 (Finite Counting Converse)**.** *Let F range over K possible values. Suppose exact resolution is attempted using an L-bit side-information tag and a decoder that must recover the correct value with zero error from that tag and a fixed observation transcript. Then*

$$K \leq 2^L.$$

Equivalently, exact recovery of K ambiguous states requires at least $\log_2 K$ bits of side information.

(L: UNQ1)

Proof sketch. There are only 2^L possible tags. In the strengthened Lean observer/tag model, exact recovery is equivalent to injectivity of the combined observation-plus-tag map (OBS1, OBS2). If the observation transcript is fixed, this reduces to injectivity of the tag map on the ambiguous states themselves. Hence two distinct values cannot share one tag under zero-error decoding, and injectivity is possible only when $K \leq 2^L$. ■

This converse formalizes the intuition used in the main text: unless the side-information channel has enough distinct tags to separate all ambiguous states, exact recovery is impossible. The modification-complexity consequence comes from the architectural fallback: when exact recovery cannot be certified from side information, systems that still require correctness must maintain or consult additional independent sources, which raises DOF and therefore update cost.

E. Information-constrained DOF lower bound

The following lemma makes that link explicit.

Lemma VI.6 (Information-constrained DOF Lower Bound)**.** *Let F range over K possible values. If the available side-information scheme exposes at most 2^L distinguishable tags with $K > 2^L$, then no zero-error resolver can recover F from those tags alone. Any architecture that nevertheless guarantees correctness must therefore retain additional independent encodings for some subset of values, hence DOF > 1 on that subset. Consequently, for facts encoded at n independent locations the effective modification complexity satisfies $M_{\text{effective}} = \Omega(n)$.*

(L: UNQ1)

Proof sketch. By Theorem VI.5, if $K > 2^L$ then the side-information tags are too small to support exact recovery of all K possibilities. The strengthened Lean architecture layer makes this explicit: every exact-recovery architecture carries a formal support count, and that support count is lower-bounded both globally by the observation/support product bound (OBS7) and locally by any confusability clique (OBS8, OBS9). In particular, observation-only architectures (support count $= 1$) are impossible on nontrivial cliques (OBS10). The new bridge theorem then identifies this support count with a concrete instance of the paper's 'Dof.dof' semantics under a derivation-free support architecture (OBS11); hence nontrivial cliques force a formal DOF strictly above one (OBS12). Therefore, if correctness is still required under updates, the architecture must expose additional independently accessible discriminating support; in the paper's DOF reading, this means moving above the single-source regime. Once DOF> 1 over a set of n independent locations, Theorem VI.3 implies $M_{\text{effective}} = \Omega(n)$. ■

F. Worked example: finite tag budget

Consider a fact F with $K = 4$ possible values. Suppose the available side-information mechanism supplies only a 1-bit tag, so at most $2^1 = 2$ distinct tags are available.

Apply Theorem VI.5. Exact recovery would require

$$4 \leq 2^1,$$

which is false. So 1 bit of side information cannot resolve the 4-way ambiguity exactly.

Interpretation: with only 2 available tags for 4 possible values, some pair of values must collide. Therefore, to guarantee exact recovery one must either supply more side information (increase L until $2^L \geq 4$) or enrich the architecture with a nontrivial auxiliary channel; in the paper's systems reading, that means additional independently accessible discriminating information, which in turn raises manual modification costs per Theorem VI.3.

By contrast, a 2-bit tag budget gives $2^2 = 4$ tags, which is just enough to distinguish all four values in principle.

The same reasoning extends to a finite relaxed-error setting where exact recovery is only required on a designated subset of states: any confusability clique that must still be recovered exactly obeys the same tag-alphabet lower bound (OBS13).

More generally, the source proof tree now contains an explicit finite distribution/error layer. For a finite source distribution with point-mass ceiling m, the total probability of exactly recovered states is bounded by $(|\mathcal{O}| \cdot |\mathcal{T}|)m$ (PRB2), so the error probability is bounded below by $1 - (|\mathcal{O}| \cdot |\mathcal{T}|)m$ (PRB3). For families of instances with a uniform budget-mass gap, this yields a fully mechanized non-vanishing-error theorem (PRB6).

For the uniform finite source, these bounds sharpen into a direct weak-Fano-style converse: success probability is at most the observation/tag budget fraction (PRB8), hence any decoder with error at most ε must satisfy

$$1-\varepsilon \le \frac{|\mathcal{O}|\,|\mathcal{T}|}{K},$$

formalized in Lean as PRB9. The strengthened source proofs also identify the exact success probability under the uniform source with the fraction of states decoded exactly (PRB11, PRB12), yielding a cardinality form of the same converse: low error forces a large exactly recovered subset (PRB13), which is then bounded by the observation/tag budget to recover the weak-Fano inequality (PRB14). Families with a uniform gap below this threshold cannot have vanishing error (PRB10).

The entropy layer now goes one step further: a fully mechanized partition identity shows that for the uniform source, the success/failure split satisfies

$$\log K = h(P_e) + (1-P_e)\log|S| + P_e \log|F|,$$

where S is the exactly recovered subset and F its complement (PRB15). Combining this identity with the exact success-set budget bound yields an explicit finite Fano-style inequality in the source proof tree:

$$\log K \le h(P_e) + (1-P_e)\log(|\mathcal{O}||\mathcal{T}|) + P_e \log(K-1)$$

for the uniform finite source with nontrivial success/failure partition (PRB16). In the observation-only specialization this reduces to a q-ary entropy bound (PRB17).

The source proofs then sharpen the observation-only case further: for a uniform K-state source, observation-only recovery succeeds with probability at most $1/K$ (PRB18) and therefore has error probability at least $1-1/K$ (PRB19).

Beyond the uniform case, the probabilistic layer now internalizes the source distribution itself through its maximum atom mass. For any finite source distribution, success probability is bounded by $(|\mathcal{O}||\mathcal{T}|)$ times the source's own max-mass parameter (PRB20), giving an intrinsic error lower bound (PRB21) and a weak-Fano-style converse without an externally supplied mass cap (PRB22). Families with a uniform budget–max-mass gap cannot achieve vanishing error (PRB23).

The same intrinsic max-mass layer also yields a structural cardinality consequence: the exactly recovered subset must have size at least $(1-\varepsilon)/m_{\max}$ whenever the error probability is at most ε (PRB24, PRB25). In the observation-only specialization, success is at most $m_{\max}$ and error is at least $1-m_{\max}$ for arbitrary finite sources (PRB26, PRB27).

Finally, the arbitrary-source converse can be rewritten in min-entropy form. Defining $H_\infty(X) = -\log m_{\max}$, the source proofs now show

$$P_{\text{succ}} \le |\mathcal{O}||\mathcal{T}|\, e^{-H_\infty(X)}, \qquad P_e \ge 1-|\mathcal{O}||\mathcal{T}|\, e^{-H_\infty(X)},$$

formalized as PRB28 and PRB29. This gives a clean min-entropy interpretation of the finite-source converse.

The min-entropy layer also yields an exact-recovery-set interpretation: if error is at most ε, then the exactly recovered subset must have size at least $(1-\varepsilon)e^{H_\infty(X)}$ (PRB32), and therefore the available observation/tag budget must satisfy the same lower bound (PRB33).

The source layer also now defines a Shannon-style finite entropy for arbitrary finite sources and proves the entropy sandwich $H_\infty(X) \le H(X)$ (PRB31), with the uniform finite source attaining $H(X) = \log K$ exactly (PRB30). This gives a clean conceptual relation between the earlier uniform-source Fano bounds and the later arbitrary-source min-entropy converses.

It also now records the finite-cardinality ceiling directly at the source level: every K-state source satisfies $1/K \le m_{\max}$ (PRB34), hence

$$H_\infty(X) \le \log K,$$

formalized as PRB35. In the observation-only specialization, the min-entropy converse collapses to the intrinsic one-state budget bound

$$P_{\text{succ}} \le e^{-H_\infty(X)}, \qquad P_e \ge 1-e^{-H_\infty(X)},$$

formalized as PRB36 and PRB37. This isolates the best possible observation-only performance directly in terms of source concentration, without introducing any auxiliary alphabet-size parameter.

The Shannon layer now closes the other side of the finite-source picture as well. The source entropy is always nonnegative (PRB38) and is upper-bounded by the ambient alphabet entropy,

$$H(X) \le \log K,$$

formalized as PRB39. Combined with the previously mechanized min-entropy lower comparison (PRB31), the source proof tree now contains the full finite entropy chain

$$H_\infty(X) \le H(X) \le \log K,$$

so the arbitrary-source converses are no longer expressed only through max-mass and min-entropy, but also situated inside the standard Shannon entropy envelope of the finite alphabet.

G. Unified Converse Viewpoint

The point of this section is that the many finite lemmas are not separate claims but one converse family. Let X be the latent source, let $Y = (O,T)$ denote the deterministic observation/tag transcript, and let $\hat{X}$ denote any deterministic decoder output. Then the source proof tree supports the following synthesis:

Theorem VI.7 (Unified finite converse, informal synthesis)**.** *If the observation transcript does not already isolate the true state, then the same finite budget simultaneously constrains*

1) *zero-error resolution of ambiguity classes and confusability cliques,*
2) *the conditional entropy $H(X \mid Y)$,*
3) *the decoder-output entropy $H(\hat{X})$,*
4) *the deterministic mutual information $I(X;Y) = H(Y)$,*
5) *and the entropy deficits $\log|\mathcal{Y}| - H(Y)$ and $\log|\hat{\mathcal{X}}| - H(\hat{X})$ via KL divergence from the corresponding uniform output laws.*

Equivalently, the paper's counting, clique, Fano-style, decoder-output, and PMF/KL statements are different normal forms of one finite converse theorem family.

Read the theorem stack in that order: exact zero-error separation first, then entropy converses, then PMF/KL saturation and gap witnesses. The same obstruction simply reappears in several classical information-theoretic languages.

The unified converse can be read in four compressed layers.

Min-entropy and Shannon envelope. The budget first appears as a min-entropy bound:

$$H_\infty(X) \le \log(|\mathcal{O}||\mathcal{T}|) - \log(1-\varepsilon),$$

formalized as PRB40, with the observation-only specialization PRB41. Together with the finite-source envelope

$$H_\infty(X) \le H(X) \le \log K$$

(PRB31, PRB38, PRB39), this places the later Fano-style converses inside a standard entropy hierarchy.

Source-side Fano and conditional entropy. For arbitrary finite sources, the source entropy obeys the exact success/failure partition bound PRB42, the budgeted Fano inequality

$$H(X) \le h(P_e) + (1-P_e)\log(|\mathcal{O}||\mathcal{T}|) + P_e \log(K-1),$$

(PRB43), and its observation-only specialization PRB44. The same family admits a conditional-entropy form: the model defines a genuine deterministic $H(X \mid Y)$ with chain rule PRB47, yields

$$H(X \mid Y) \le h(P_e) + (1-P_e)\log(|\mathcal{O}||\mathcal{T}|) + P_e \log(K-1)$$

(PRB50), and specializes to observation-only as PRB51. The graph layer meets this Shannon layer through clique-sensitive entropy bounds PRB54 and PRB55.

Observation-side mutual information and KL gaps. The same model identifies deterministic mutual information with output entropy,

$$I(X;Y) = H(Y) = H(X) - H(X \mid Y),$$

formalized by PRB49, PRB57, and PRB58. The same layer gives the budget and source ceilings PRB52, PRB53, PRB60, PRB62, and PRB63, deterministic data processing under coarsenings PRB64–PRB68, and a PMF/KL saturation story: zero KL to the uniform output law forces $H(Y) = I(X;Y) = \log(|\mathcal{O}||\mathcal{T}|)$ (PRB69–PRB76, ENT10–ENT12).

Decoder-output and success/failure forms. The success/failure indicator is itself a finite pushforward source and Bernoulli PMF with entropy exactly $h(P_e)$ (PRB56, PRB77, PRB79, ENT9). The decoder output obeys the same converse program:

$$H(\hat{X}) \le I(X;Y) \le H(X),$$

with explicit source and alphabet gaps PRB81–PRB83, a decoder-native Fano bound

$$H(\hat{X}) \le h(P_e) + (1-P_e)\log(|\mathcal{O}||\mathcal{T}|) + P_e \log(K-1)$$

(PRB84, PRB85, PRB87), with its own PMF/KL saturation story on 'Fin$(K+1)$' (PRB91–PRB95, ENT13). The observable/random-variable wrappers simply package these deterministic pushforwards in a reusable interface.

Tightness (Achievability + Converse). Theorems VI.2 and VI.3 together show that the operational cost transition at rate 1 is sharp: rate 1 achieves constant manual cost, while any architecture with more than one independent location incurs linear cost in the number of independent encodings (UNQ1, UNQ1).

H. The Unbounded Gap

Theorem VI.8 (Unbounded Gap)**.** *The ratio of modification complexity between DOF-1-incomplete and DOF-1-complete architectures grows without bound:*

$$\lim_{n\to\infty} \frac{M_{DOF>1}(n)}{M_{DOF=1}} = \lim_{n\to\infty} \frac{n}{1} = \infty$$

(L: UNQ1)

Proof. By Theorem VI.2, $M_{\text{DOF}=1} = O(1)$. Specifically, $M_{\text{DOF}=1} = 1$ for any system size.

By Theorem VI.3, $M_{\text{DOF}>1}(n) = \Omega(n)$ where n is the number of independent encoding locations.

The ratio is:

$$\frac{M_{\text{DOF}>1}(n)}{M_{\text{DOF}=1}} = \frac{n}{1} = n$$

As $n \to \infty$, the ratio $\to \infty$. The gap is unbounded. ■

Corollary VI.9 (Arbitrary Reduction Factor)**.** *For any constant k, there exists a system size n such that DOF = 1 provides at least $k\times$ reduction in modification complexity.*

(L: UNQ1)

Proof. Choose $n = k$. Then $M_{\text{DOF}>1}(n) = n = k$ and $M_{\text{DOF}=1} = 1$. The reduction factor is $k/1 = k$. ■

I. The (R, C, P) Tradeoff Space

We now summarize the tradeoff space induced by the model.

Definition VI.10 ((R, C, P) Tradeoff)**.** For an encoding system, define:

- R = *Rate* (DOF): Number of independent encoding locations
- C = *Complexity*: Manual modification cost per change
- P = *Coherence indicator*: $P = 1$ iff no incoherent state is reachable; otherwise $P = 0$

The *(R, C, P) tradeoff space* is the set of achievable (R, C, P) tuples.

Theorem VI.11 (Operating Regimes)**.** *The (R, C, P) space has three distinct operating regimes:*

Rate	***Complexity***	***Coherence***	***Interpretation***
$R = 0$	$C = 0$	$P =$ *undefined*	*Fact not encoded*
$R = 1$	$C = O(1)$	$P = 1$	***Optimal (capacity-achieving)***
$R > 1$	$C = \Omega(R)$	$P = 0$	*Above capacity*

(L: UNQ1)

Proof. $R = 0$**:** No encoding exists. Complexity is zero (nothing to modify), but coherence is undefined (nothing to be coherent about).

$R = 1$**:** By Theorem VI.2, $C = O(1)$. By Theorem II.37, $P = 1$ (coherence guaranteed). This is the capacity-achieving regime.

$R > 1$**:** By Theorem VI.3, $C = \Omega(R)$. By Theorem II.8, incoherent states are reachable, so $P = 0$. ■

Definition VI.12 (Pareto Frontier)**.** A point (R, C, P) is *Pareto optimal* if no other achievable point dominates it (lower R, lower C, or higher P without worsening another dimension).

The *Pareto frontier* is the set of all Pareto optimal points.

Theorem VI.13 (Pareto Optimality of DOF = 1)**.** *$(R = 1, C = 1, P = 1)$ is the unique Pareto optimal point for encoding systems requiring coherence ($P = 1$).*

(L: UNQ1)

Proof. We show $(1, 1, 1)$ is Pareto optimal and unique:

Existence: By Theorems VI.2 and II.37, the point $(1, 1, 1)$ is achievable.

Optimality: Consider any other achievable point (R', C', P') with $P' = 1$:

- If $R' = 0$: Fact is not encoded (excluded by requirement)
- If $R' = 1$: Same as $(1, 1, 1)$ (by uniqueness of C at $R = 1$)
- If $R' > 1$: By Theorem II.8, $P' < 1$, contradicting $P' = 1$

Uniqueness: No other point achieves $P = 1$ except $R = 1$. ■

Information-theoretic interpretation. If one uses $1 - P$ as a coarse distortion surrogate, then the zero-distortion boundary collapses to a single point: $R = 1$ is the only rate achieving perfect coherence.

Corollary VI.14 (No Tradeoff at $P = 1$)**.** *Unlike rate-distortion where you can trade rate for distortion, there is no tradeoff at $P = 1$ (perfect coherence). The only option is $R = 1$.*

(L: UNQ1)

Proof. Direct consequence of Theorem II.37. ■

Comparison to rate-distortion. In rate-distortion theory:

- You can achieve lower distortion with higher rate (more bits)
- The rate-distortion function $R(D)$ is monotonically decreasing
- $D = 0$ (lossless) requires $R = H(X)$ (source entropy)

In our framework:

- You *cannot* achieve higher coherence (P) with more independent locations
- Higher rate ($R > 1$) *eliminates* coherence guarantees ($P = 0$)
- $P = 1$ (perfect coherence) requires $R = 1$ exactly

The key difference is that extra independent representations increase ambiguity unless they are coordinated by derivation. In this setting, uncontrolled redundancy hurts exact consistency.

J. Practical Implications

The unbounded gap has practical implications, but these should be read as consequences of the abstract rate/computation result rather than as the main theorem of the paper:

1. DOF = 1 matters more at scale. For small systems ($n = 3$), the difference between 3 edits and 1 edit is minor. For large systems ($n = 50$), the difference between 50 edits and 1 edit is significant.

2. The gap compounds over time. Each modification to fact F incurs the complexity cost. If F changes m times over the system lifetime, total cost is $O(mn)$ with DOF > 1 vs. $O(m)$ with DOF = 1.

3. The gap affects error rates. Each manual edit is an opportunity for error. With n edits, the probability of at least one error is $1-(1-p)^n$ where p is the per-edit error probability. As n grows, this approaches 1.

Example VI.15 (Error Rate Calculation)**.** Assume a 1% error rate per edit ($p = 0.01$).

Edits (n)	P(at least one error)	Architecture
1	1.0%	DOF = 1
10	9.6%	DOF = 10
50	39.5%	DOF = 50
100	63.4%	DOF = 100

With 50 independent encoding locations (DOF = 50), there is a 39.5% chance of introducing an error when modifying fact F. With DOF = 1, the chance is 1%.

K. Amortized Analysis

The complexity bounds assume a single modification. Over the lifetime of an encoding system, facts are modified many times (UNQ1).

Theorem VI.16 (Amortized Complexity)**.** *Let fact F be modified m times over the system lifetime. Let n be the number of independent encoding locations. Total modification cost is:*

- *DOF = 1: $O(m)$*
- *DOF = $n > 1$: $O(mn)$*

(L: UNQ1)

Proof. Each modification costs $O(1)$ with DOF = 1 and $O(n)$ with DOF = n. Over m modifications, total cost is $m \cdot O(1) = O(m)$ with DOF = 1 and $m \cdot O(n) = O(mn)$ with DOF = n. ■

For a fact modified 100 times with 50 independent encoding locations:

- DOF = 1: 100 edits total
- DOF = 50: 5,000 edits total

The 50× reduction factor applies to every modification, compounding over the system lifetime.

VII. Application: Realizability Patterns (Worked Example)

This section contained an extended worked example and multiple detailed code snippets illustrating realizability patterns from the OpenHCS case study. For the revised IT-first manuscript we move the full worked example, all code excerpts, and the detailed DOF measurements to Supplement A ("Practical Patterns and Case Study").

Below we give only a short summary and pointer to the supplement; the complete instantiation, PR traces, and verifiable diffs are available in Supplement A.

Summary (short):

- Four recurring patterns realize DOF = 1: contract enforcement, automatic registration, automatic discovery, and introspection-driven code generation.
- A publicly verifiable OpenHCS pull request (PR 44) demonstrates a 47→1 DOF reduction; full diffs and measurements are in Supplement A.
- The worked examples illustrate why both definition-time hooks and runtime introspection are necessary for DOF = 1; language-by-language impossibility notes are in Supplement A.

For the interested reader, Supplement A contains the complete before/after code, the methodology, measurement tables, and links to the public PR and repository snapshots.

VIII. Related Work

This section focuses on the information-theoretic lineage. Application-side references appear only where they clarify realizability.

A. Zero-Error and Multi-Terminal Source Coding

Our exact-consistency threshold belongs to the intersection of zero-error/confusability theory, side-information source coding, and finite Fano-style converses.

Zero-Error Capacity. Shannon [1] introduced zero-error capacity as exact communication under a confusability constraint; Körner [2] and Lovász [3] developed the graph-theoretic language for that regime. Our zero-incoherence capacity is a deterministic storage/update analogue: the confusable objects are reachable multi-location states, and exact recovery means resolving them with zero ambiguity. Ambiguity classes and cliques in our model play the role of confusability classes in zero-error communication, while the threshold $C_0 = 1$ is the storage-side counterpart of a sharp zero-error feasibility boundary.

Multi-Terminal Source Coding and Fano. Slepian and Wolf [4] characterized distributed encoding with decoder side information, while classical Fano inequalities upper-bound $H(X \mid Y)$ in terms of error probability. Here the side information is not a second stochastic source, but a deterministic transcript produced by a modifiable multi-location encoding. The resulting converse is therefore a deterministic finite-source analogue: the same observation/tag budget controls zero-error resolution, $H(X \mid Y)$, and the entropy of the decoder output. This is closer to finite exact-resolvability with side information than to ordinary vanishing-error distributed compression.

Multi-Version Coding and Interaction. Rashmi et al. [12] quantify storage cost under consistency constraints; Ma and Ishwar [10] and the BIRS interactive-IT agenda [9] show that interaction can change information requirements. Our setting differs in that the object being constrained is a modifiable encoding itself. That turns exact consistency into a combined zero-error, side-information, and operational-complexity problem.

Classical Source Coding and Rate-Distortion. Shannon [19] and Slepian–Wolf [4] provide the source-coding background, while Cover and Thomas [11] supply the standard entropy/rate language. We use that language in a deterministic finite model: provenance observability acts as side information, derivation as perfect dependence, and independent rate governs modification cost rather than ordinary distortion.

a) Closest prior work and novelty.: Prior work supplies the ingredients separately: zero-error/confusability, side-information source coding, Fano inequalities, and consistency-constrained storage. The novelty here is to make them one theorem family in a model with reachable updates: the same exact-consistency obstruction appears as clique separation, a side-information lower bound, a conditional-entropy converse, a decoder-output converse, and a PMF/KL output-gap witness.

B. Distributed Systems Consistency

Distributed-systems impossibility results provide useful structural analogies. In particular, our CAP/FLP-style statements isolate how independent updates create ambiguity and how ambiguity blocks internally justified resolution without side information.

C. Computational Reflection and Metaprogramming

Metaobject protocols and reflection. Kiczales et al. [20] and Smith [21] provide the classical foundations for systems that can execute code at definition time and introspect their own structure. These mechanisms correspond directly to causal propagation and provenance observability in our realizability theorem, explaining why MOP-equipped languages admit DOF = 1 for structural facts.

D. Application-Level Design Principles

Application-level design principles such as DRY [8], information hiding [22], and duplication analyses [23], [24] motivate coherence and single-source design. These links are interpretive only; the proofs do not rely on them.

E. Formal Methods

Our Lean 4 [13] formalization follows the tradition of mechanized theory (e.g., Pierce [25], Winskel [26], CompCert [27]), but applies it to an information-theoretic encoding model.

F. Novelty and IT Contribution

To our knowledge, this is the first work to:

1) **Characterize the exact-consistency threshold**–$C_0 = 1$ exactly, with explicit achievability and converse.

2) **Quantify side information for resolution**–$\geq \log_2 k$ bits for a k-way ambiguity class (Theorem II.48).
3) **Characterize realizability**–causal propagation and provenance observability are necessary and sufficient to realize the threshold in concrete systems (Theorem IV.9).
4) **Establish rate-complexity consequences**–$O(1)$ at rate 1 versus $\Omega(n)$ above it, with unbounded gap (Theorem VI.8).

What is new: the modifiable multi-location encoding setting, the exact-consistency threshold for that setting, and the unified converse family tying together ambiguity classes, confusability cliques, deterministic side information, Fano-style entropy bounds, decoder-output entropy bounds, and PMF/KL output-gap witnesses. The application instantiations are corollaries.

IX. Conclusion

We have studied exact consistency in interactive multi-location encoding systems through an information-theoretic lens. The main conclusion is simple: once ambiguity survives the observation transcript, exact consistency becomes a unified converse problem.

The paper contributes three main claims. First, a k-way ambiguity class requires at least $\log_2 k$ bits of side information, and the same finite budget reappears as a conditional-entropy/Fano bound, a decoder-output entropy bound, and an output-side PMF/KL gap witness. Second, zero-incoherence capacity is exactly 1: one independent source can support arbitrarily many derived views, but any higher independent rate creates reachable ambiguity. Third, this threshold has an operational consequence: modification complexity is $O(1)$ at rate 1 and $\Omega(n)$ above it, with unbounded gap. The realizability theorem identifies when the capacity-achieving regime is attainable: causal propagation and provenance observability are both necessary and sufficient.

The broader IT point is that the same obstruction admits several classical descriptions at once: zero-error/confusability structure, side-information insufficiency, conditional-entropy and decoder-output Fano converses, and KL-detectable output non-uniformity. That unification is the main claim of the paper.

Implications:

1) **For information theorists:** Zero-error, graph-confusability, Fano-style, and KL-gap ideas extend naturally to interactive multi-location encoding with modification constraints. The novelty is that one exact-consistency obstruction admits all of these normal forms in one deterministic finite model.
2) **For coding theorists:** Derivation is the mechanism that introduces dependence and reduces effective encoding rate. The realizability theorem characterizes when that regime is attainable, while the unified converse identifies what fails when independent rate exceeds the threshold.
3) **For system designers:** If exact consistency is required, independent rate must be ≤ 1. Above that threshold, correctness requires external side information or additional operational cost.

Limitations:

- Results apply primarily to facts with modification constraints. Streaming data and high-frequency updates have different characteristics.
- The complexity bounds are asymptotic. For small encoding systems (DOF < 5), the asymptotic gap is numerically small.
- Application examples are brief and primarily computational. Richer non-software case studies are left to future work.
- Realizability requirements focus on computational systems. Physical and biological encoding systems require separate analysis.

Future Work:

- **Probabilistic coherence:** Extend the present finite deterministic converse program to soft constraints where incoherence probability is bounded but non-zero.
- **Network encoding:** Study coherence capacity in networked encoding systems with communication constraints, connecting to network information theory.
- **Rate-complexity extension:** Characterize the full rate-complexity function $R(M)$ beyond the sharp zero-error boundary analyzed here.
- **Interactive capacity:** Study capacity under multi-round modification protocols, connecting to interactive information theory and directed information.
- **Partial correlation:** Characterize coherence guarantees when derivation introduces partial (non-deterministic) correlation, moving closer to classical stochastic side-information models.

A. Artifacts

The Lean 4 formalization is included as supplementary material [28]. The OpenHCS case study and associated code references are provided for the worked instantiation (Section VII).

References

[1] C. E. Shannon, "Zero-error capacity of a noisy channel," *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 8–19, 1956, introduces zero-error capacity; fundamental limit for error-free communication over noisy channels.

[2] J. Körner, "Coding of an information source having ambiguous alphabet and the entropy of graphs," *Transactions of the 6th Prague Conference on Information Theory*, pp. 411–425, 1973, zero-error source coding; graph entropy characterizes zero-error capacity.

[3] L. Lovász, "On the shannon capacity of a graph," *IEEE Transactions on Information Theory*, vol. 25, no. 1, pp. 1–7, 1979, theta function bounds Shannon capacity; foundational for zero-error information theory.

[4] D. Slepian and J. K. Wolf, "Noiseless coding of correlated information sources," *IEEE Transactions on Information Theory*, vol. 19, no. 4, pp. 471–480, 1973, distributed source coding with side information; decoder uses correlated information to reduce required rate.

[5] E. A. Brewer, "Towards robust distributed systems," in *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing*. ACM, 2000, keynote introducing CAP conjecture: Consistency, Availability, Partition tolerance—pick two.

[6] J. J. Hunt, K.-P. Vo, and W. F. Tichy, "Delta algorithms: An empirical analysis," *ACM Transactions on Software Engineering and Methodology*, vol. 7, no. 2, pp. 192–214, 1998, delta compression for version control; coherence under branch divergence.

[7] T. Delaet, W. Joosen, and J. Van de Craen, "A survey of system configuration tools," *Proceedings of the 24th Large Installation System Administration Conference (LISA)*, 2010, survey of configuration management systems; coherence challenges in multi-file configurations.

[8] A. Hunt and D. Thomas, *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley Professional, 1999.

[9] Banff International Research Station, "Interactive information theory (12w5119)," Workshop Report, 2012, workshop on interactive IT: source coding, channel coding, directed information; https://www.birs.ca/events/2012/5-day-workshops/12w5119.

[10] N. Ma and P. Ishwar, "Some results on distributed source coding for interactive function computation," *IEEE Transactions on Information Theory*, vol. 57, no. 9, pp. 6180–6195, 2011, interactive distributed source coding; shows interaction can reduce rate for function computation.

[11] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006, comprehensive textbook covering rate-distortion theory, source coding, channel coding.

[12] K. V. Rashmi, N. B. Shah, K. Ramchandran, and D. Gu, "Multi-version coding—an information-theoretic perspective of consistent distributed storage," *IEEE Transactions on Information Theory*, vol. 63, no. 6, pp. 4111–4128, 2017, information-theoretic framework for consistent distributed storage; proves inevitable storage cost for consistency guarantees.

[13] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*. Springer, 2021, pp. 625–635.

[14] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002, formal proof of CAP theorem.

[15] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985, fLP impossibility: deterministic consensus impossible in async systems with one failure.

[16] R. L. Rivest and A. Shamir, "How to reuse a "write-once" memory," in *Proceedings of the 14th Annual ACM Symposium on Theory of Computing*. ACM, 1982, pp. 105–113, foundational paper on write-once memory codes; capacity for t writes is $\log_2(t+1)$ bits per cell.

[17] J. K. Wolf *et al.*, "Coding for a write-once memory," *AT&T Bell Laboratories Technical Journal*, vol. 63, no. 6, pp. 1089–1112, 1984, extends WOM codes; establishes capacity results for irreversible storage.

[18] TIOBE Software BV, "TIOBE index for programming languages," TIOBE Programming Community Index, 2024, online: tiobe.com/tiobe-index/.

[19] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948, foundational paper establishing information theory; source coding and channel capacity.

[20] G. Kiczales, J. des Rivières, and D. G. Bobrow, *The Art of the Metaobject Protocol*. MIT press, 1991.

[21] B. C. Smith, "Reflection and semantics in lisp," in *Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1984, pp. 23–35.

[22] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[23] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 1999.

[24] C. K. Roy and J. R. Cordy, "A survey on software clone detection research," *School of Computing TR 2007-541, Queen's University*, vol. 115, pp. 64–68, 2007.

[25] B. C. Pierce, *Types and programming languages*. MIT press, 2002.

[26] G. Winskel, *The Formal Semantics of Programming Languages: An Introduction*. MIT press, 1993.

[27] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[28] T. Simas, "Lean 4 formalization: SSOT requirements theorems," Supplementary material, 2025, 9,369 lines across 29 files, 441 theorems, 0 `sorry` placeholders. Included with paper submission.

Appendix

Mechanized Proofs (Lean 4)

Core theorem chains in this paper are machine-checked in Lean 4. The current build statistics are: **14898 lines across 60 files, 703 theorem/lemma statements, 0 `sorry` placeholders.**

Mechanization Contract

Each theorem-level handle is typed as one of:

- **Full:** direct Lean theorem support.
- **Full (derived/model-level):** theorem-level closure by composition from mechanized cores in the stated model.

a) Finite zero-error information handles (fully mechanized).:

- UNQ1 (the `sideInfoBits` quantity realizing the paper's $\log_2 k$ expression)
- UNQ1 (direct finite lower bound for side-information requirements)
- UNQ1 (finite zero-error counting converse $K \leq 2^L$)
- UNQ1 (overflow contradiction bridge from counting converse)
- OBS1 (exact recovery implies injectivity of the observation-plus-tag map)
- OBS2 (injective observation-plus-tag maps admit exact decoders)
- OBS3 (each observation fiber is bounded by the tag alphabet)
- OBS4 (global observer/tag counting converse $|V| \leq |O| \cdot |T|$)
- OBS5 (confusability cliques require tag alphabets at least as large as the clique)
- OBS6 (nontrivial cliques rule out observation-only exact recovery)
- OBS7 (every exact-recovery architecture obeys a global support-count bound)
- OBS8 (confusability cliques lower-bound formal architecture support count)
- OBS9 (nontrivial cliques force architecture support count > 1)
- OBS10 (observation-only architectures are impossible on nontrivial cliques)
- OBS11 (formal architecture support count is realized as a concrete 'Dof.dof' quantity)
- OBS12 (nontrivial cliques force that concrete DOF strictly above one)
- OBS13 (finite relaxed-error extension: any clique recovered exactly on a subset still obeys the tag lower bound)
- PRB1 (exactly recovered state set has cardinality at most the observation/tag budget)
- PRB2 (success probability is bounded by budget times a point-mass ceiling)
- PRB3 (error probability lower bound from finite budget and point-mass ceiling)
- PRB4 (elementary definition of vanishing error)
- PRB5 (uniform positive error lower bound rules out vanishing error)
- PRB6 (finite-family nonvanishing theorem under a uniform budget/mass gap)
- PRB7 (uniform-source error lower bound from observation/tag budget)
- PRB8 (uniform-source success probability is bounded by the observation/tag budget fraction)
- PRB9 (weak-Fano-style budget lower bound for uniform finite sources)

- PRB10 (uniform-source families with a budget-ratio gap cannot have vanishing error)
- PRB11 (under the uniform source, success probability equals the exactly recovered fraction)
- PRB12 (under the uniform source, error probability is one minus that fraction)
- PRB13 (weak-Fano cardinality form: low error forces a large exactly recovered subset)
- PRB14 (weak-Fano budget form recovered from the exact cardinality identity)
- PRB15 (uniform-source success/failure partition entropy identity)
- PRB16 (explicit finite Fano-style inequality for the uniform finite source)
- PRB17 (observation-only specialization as a ‘q‘-ary entropy bound)
- PRB18 (observation-only success probability is at most $1/K$ for the uniform source)
- PRB19 (observation-only error probability is at least $1-1/K$ for the uniform source)
- PRB20 (arbitrary-source success probability is bounded by budget times intrinsic max atom mass)
- PRB21 (arbitrary-source intrinsic error lower bound from budget and max atom mass)
- PRB22 (weak-Fano-style converse for arbitrary finite sources via intrinsic max atom mass)
- PRB23 (finite-family nonvanishing theorem using intrinsic max atom mass)
- PRB24 (exactly recovered subset cardinality is bounded below by success probability over max atom mass)
- PRB25 (low error forces a large exactly recovered subset via intrinsic max atom mass)
- PRB26 (observation-only success is at most max atom mass for arbitrary finite sources)
- PRB27 (observation-only error is at least one minus max atom mass for arbitrary finite sources)
- PRB28 (arbitrary-source success bound rewritten in min-entropy form)
- PRB29 (arbitrary-source error lower bound rewritten in min-entropy form)
- PRB30 (uniform finite source has Shannon entropy exactly $\log K$)
- PRB31 (min-entropy is bounded above by the Shannon-style finite entropy)
- PRB32 (low error forces an exactly recovered subset of size at least $(1-\varepsilon)e^{H_\infty}$)
- PRB33 (the observation/tag budget must dominate that min-entropy lower bound)
- PRB34 (every finite K-state source has max atom mass at least $1/K$)
- PRB35 (finite-source min-entropy is bounded above by $\log K$)
- PRB36 (observation-only success is at most $e^{-H_\infty(X)}$)
- PRB37 (observation-only error is at least $1-e^{-H_\infty(X)}$)
- PRB38 (Shannon-style finite source entropy is always nonnegative)
- PRB39 (Shannon-style finite source entropy is bounded above by $\log K$)
- PRB40 (low error forces an explicit log-budget upper bound on source min-entropy)
- PRB41 (observation-only recovery forces $H_\infty(X) \leq -\log(1-\varepsilon)$)
- PRB42 (arbitrary-source entropy is bounded by the success/failure partition entropy plus support logs)
- PRB43 (finite-alphabet Fano-style inequality for arbitrary finite sources)
- PRB44 (observation-only arbitrary-source specialization as a ‘q‘-ary entropy bound)
- PRB45 (conditional-style rearrangement of the arbitrary-source Fano bound)
- PRB46 (observation-only conditional-style Fano bound for arbitrary finite sources)
- PRB47 (native deterministic chain rule $H(X) = H(Y) + H(X \mid Y)$ for the induced observation/tag variable)
- PRB48 (the induced conditional entropy is bounded above by the source entropy)
- PRB49 (the finite mutual-information surrogate equals the induced observation/tag entropy)
- PRB50 (literal conditional-entropy Fano bound for the induced observation/tag variable)
- PRB51 (observation-only conditional-entropy Fano specialization)
- PRB52 (the induced observation/tag entropy is bounded by $\log(|\mathcal{O}||\mathcal{T}|)$)
- PRB53 (the finite mutual-information surrogate obeys the same log-budget ceiling)
- PRB54 (any exactly recovered confusability clique has weighted entropy contribution bounded by tag budget)
- PRB55 (the decoder success set inherits that clique-entropy bound when it is clique-like)
- PRB56 (the induced success/failure PMF has entropy exactly $h(P_e)$)
- PRB57 (deterministic finite mutual information equals induced observation/tag entropy)
- PRB58 (the same deterministic mutual information equals $H(X) - H(X \mid Y)$)
- PRB59 (deterministic finite mutual information is nonnegative)
- PRB60 (deterministic finite mutual information is bounded by $\log(|\mathcal{O}||\mathcal{T}|)$)
- PRB61 (the induced conditional entropy $H(X \mid Y)$ is nonnegative)
- PRB62 (the induced observation/tag entropy is bounded above by the source entropy)
- PRB63 (deterministic finite mutual information is also bounded above by source entropy)
- PRB64 (deterministic coarsenings of the observation/tag variable cannot increase induced entropy)
- PRB65 (coarsened deterministic mutual information still equals coarsened observation entropy)
- PRB66 (deterministic coarsenings satisfy a finite data-processing-style monotonicity law)
- PRB67 (the same data-processing law is packaged directly as a deterministic-kernel mutual-information theorem)
- PRB68 (the deterministic-kernel entropy form is also

explicit)
- PRB69 (KL-zero to uniform output forces the induced output PMF itself to be uniform)
- PRB70 (that KL-zero uniform case forces observation entropy to saturate the log-budget ceiling)
- PRB71 (the same KL-zero case forces deterministic mutual information to saturate the log-budget ceiling)
- PRB72 (any non-uniform induced output PMF yields nonzero KL divergence from uniform)
- PRB73 (the observation/tag entropy deficit from the log-budget ceiling is always nonnegative)
- PRB74 (zero KL divergence to uniform collapses that output-entropy deficit to zero)
- PRB75 (a strictly positive output-entropy deficit forces nonzero KL divergence from uniform)
- PRB76 (the same strict-gap witness is available for deterministic mutual information)
- PRB77 (the success/failure pushforward source has entropy exactly $h(P_e)$)
- PRB78 (that success/failure entropy is bounded above by source entropy via finite pushforward monotonicity)
- PRB79 (the same $h(P_e)$ fact is exposed through the generic finite-random-variable interface)
- PRB80 (the success/failure random-variable entropy obeys the same source-entropy upper bound)
- PRB81 (the decoder-output random-variable entropy is bounded above by deterministic mutual information)
- PRB82 (the decoder-output entropy deficit relative to source entropy is nonnegative)
- PRB83 (the decoder-output entropy deficit relative to its own finite alphabet budget is nonnegative)
- PRB84 (decoder-output entropy satisfies a success/failure partition bound with exact recovery-set terms)
- PRB85 (decoder-output entropy satisfies a budgeted Fano-style converse)
- PRB86 (the success/failure observable wrapper still has entropy exactly $h(P_e)$)
- PRB87 (observation-only specialization of the decoder-native Fano bound)
- PRB88 (the success/failure observable wrapper also inherits the source-entropy upper bound)
- PRB89 (the decoder-output observable is a coarsening of the observation/tag observable)
- PRB90 (the observable API exposes a generic coarsening-to-source-entropy monotonicity rule)
- PRB91 (KL-zero to uniform decoder-output PMF forces the decoder-output PMF itself to be uniform)
- PRB92 (that KL-zero decoder-output case forces $H(\hat{X}) = \log(K+1)$)
- PRB93 (zero KL divergence collapses the decoder-output alphabet-gap to zero)
- PRB94 (a strictly positive decoder-output alphabet-gap forces nonzero KL divergence)
- PRB95 (the decoder-output observable inherits the generic source-entropy upper bound)
- ENT2 (binary entropy is directly available from imported mathlib)
- ENT3 (KL divergence self-equality theorem is directly available on PMF-derived measures)
- ENT4 (PMF-level Bernoulli entropy agrees with mathlib's binary entropy)
- ENT5 (Bernoulli PMF entropy is bounded by $\log 2$)
- ENT6 (uniform PMF entropy on 'Fin(n+1)' equals $\log(n+1)$)
- ENT7 (zero KL divergence to the uniform PMF forces the PMF itself to be uniform)
- ENT8 (that KL-zero uniform case attains entropy exactly $\log K$)
- ENT9 (real-valued Bernoulli parameters can be turned directly into PMF entropy equalities)
- ENT10 (the induced observation/tag PMF entropy agrees exactly with the native observation/tag entropy)
- ENT11 (the '$\mathrm{Fin}(|\mathcal{O}||\mathcal{T}|)$' output PMF entropy agrees with the native observation/tag entropy)
- ENT12 (KL-zero to uniform on any nonempty finite alphabet forces entropy to equal $\log|\mathcal{Y}|$)
- ENT13 (the induced decoder-output PMF entropy agrees exactly with the native decoder-output entropy)

The source proof environment also now imports the current mathlib entropy/KL/PMF stack successfully via 'Ssot/EntropyGeneral.lean'. Beyond import validation, it now exposes a concrete KL-to-uniform witness for entropy maximization on finite alphabets, complementing the source-tree Fano converses.

b) Graph/confusability handles (fully mechanized finite zero-error layer).:

- UNQ1 (proper coloring bounds clique size by alphabet size)
- UNQ1 (complete confusability graph forces one tag per state)
- UNQ1 (observation-fiber clique lower bound for zero-error tagging)
- UNQ1 (constant-observation / maximal-barrier lower bound)
- UNQ1 (worst-case observation-fiber lower bound)
- UNQ1 (average observation-fiber occupancy lower bound)
- UNQ1 (global counting converse $|V| \le |O| \cdot |T|$)

Complete Claim Coverage Matrix

c) Foundations, capacity, and side-information chain.:

- thm:oracle-arbitrary: Full – UNQ1.
- thm:dof-one-coherence: Full – UNQ1.
- thm:dof-gt-one-incoherence: Full – UNQ1.
- cor:coherence-forces-ssot: Full – UNQ1.
- thm:correctness-forcing: Full – UNQ1.
- thm:dof-inconsistency: Full – UNQ1.
- cor:dof-risk: Full – UNQ1.
- thm:dof-optimal: Full – UNQ1, UNQ1.
- thm:cap-encoding: Full – UNQ1.
- thm:static-flp: Full – UNQ1.
- thm:redundancy-incoherence: Full – UNQ1.
- thm:coherence-capacity: Full – UNQ1.
- thm:capacity-achievability: Full – UNQ1.
- thm:capacity-converse: Full – UNQ1.
- cor:capacity-unique: Full – L1.
- cor:redundancy-above: Full – UNQ1.
- thm:rate-incoherence-step: Full – UNQ1.
- thm:design-necessity: Full – UNQ1.
- cor:architectural-forcing: Full – L1, UNQ1.
- thm:incoherence-graph: Full (derived/model-level) – UNQ1, UNQ1.
- thm:side-info: Full – UNQ1.

- cor:dof1-zero-side: Full – UNQ1.
- cor:side-info-redundancy: Full – UNQ1.
- cor:multi-terminal: Full (derived/model-level) – UNQ1, UNQ1.
- thm:fano-converse: Full (finite zero-error counting form) – UNQ1.
- lem:info-dof: Full (finite counting contradiction form) – UNQ1.

d) Derivation and realizability chain.:

- thm:derivation-lattice: Full (model-level) – UNQ1.
- thm:dof-antimonotone: Full (model-level) – UNQ1, UNQ1.
- cor:minimal-dof1: Full (model-level) – UNQ1, UNQ1.
- thm:dof-complexity: Full (model-level) – UNQ1.
- thm:ssot-determinate: Full – UNQ1.
- thm:ssot-optimal: Full – UNQ1.
- thm:ssot-unique: Full – L1.
- cor:no-redundancy: Full – UNQ1.
- thm:derivation-excludes: Full – UNQ1.
- cor:metaprogramming: Full (derived/model-level) – UNQ1, UNQ1.
- lem:confusability-clique: Full (finite zero-error graph layer) – UNQ1, UNQ1, UNQ1, UNQ1, UNQ1.
- thm:timing-forces: Full – UNQ1.
- thm:causal-necessary: Full – UNQ1.
- thm:provenance-necessary: Full – UNQ1.
- thm:independence: Full – UNQ1, UNQ1.
- thm:ssot-iff: Full – UNQ1.
- thm:generated-second: Full – UNQ1.
- cor:lang-realizability: Full – UNQ1.

e) Bounds, regimes, and optimization chain.:

- thm:upper-bound: Full – UNQ1.
- thm:lower-bound: Full – UNQ1.
- thm:unbounded-gap: Full – UNQ1.
- cor:arbitrary-reduction: Full – UNQ1.
- thm:operating-regimes: Full – UNQ1.
- thm:pareto-optimal: Full – UNQ1.
- cor:no-tradeoff: Full – UNQ1.
- thm:amortized: Full – UNQ1.

Verification Instructions

1) Lean-only verification (recommended): `cd docs/papers/paper2_ssot/proofs && lake build`
2) Direct Lean build: `python3 scripts/build_papers.py lean paper2_it`

Scope Note

The current source proof tree is axiom-free with respect to the paper-specific information-theoretic closure layer: the side-information and counting-converse handles are discharged by finite zero-error arguments rather than paper-local assumption bundles.

Lean Handle Index.

ENT2 ObserverModel.binEntropy_half
ENT3 ObserverModel.klDiv_self_eq_zero
ENT4 ObserverModel.pmfEntropy_bernoulli
ENT5 ObserverModel.pmfEntropy_bernoulli_le_log_two
ENT6 ObserverModel.pmfEntropy_uniform_fin
ENT7 ObserverModel.pmf_eq_uniform_fin_of_klDiv_zero
ENT8 ObserverModel.pmfEntropy_eq_log_card_of_klDiv_zero_uniform_fin
ENT9 ObserverModel.pmfEntropy_bernoulli_ofReal
ENT10 ObserverModel.pmfEntropy_inducedPairPMF_eq_observationTagEntropy
ENT11 ObserverModel.pmfEntropy_inducedPairPMFFin_eq_observationTagEntropy
ENT12 ObserverModel.pmfEntropy_eq_log_card_of_klDiv_zero_uniform_fin_nonempty
ENT13 ObserverModel.pmfEntropy_inducedDecodedOutputPMFFin_eq_decodedOutputEntropy
L1 matroid_basis_equicardinality
L2 external_tools_not_derivation
OBS1 ObserverModel.exact_recovery_implies_pair_injective
OBS2 ObserverModel.pair_injective_implies_exact_recovery
OBS3 ObserverModel.fiber_card_le_tag_alphabet
OBS4 ObserverModel.exact_recovery_global_count
OBS5 ObserverModel.clique_card_le_tag_alphabet
OBS6 ObserverModel.no_observation_only_exact_recovery_of_nontrivial_clique
OBS7 ObserverModel.architecture_support_global_bound
OBS8 ObserverModel.architecture_support_bound_from_clique
OBS9 ObserverModel.architecture_support_gt_one_of_nontrivial_clique
OBS10 ObserverModel.observation_only_architecture_impossible_on_nontrivial_clique
OBS11 ObserverModel.supportCount_eq_dof
OBS12 ObserverModel.nontrivial_clique_forces_dof_gt_one
OBS13 ObserverModel.exactOn_clique_card_le_tag_alphabet
PRB1 ObserverModel.successSet_card_le_budget
PRB2 ObserverModel.successProb_le_budget_mul_massBound
PRB3 ObserverModel.errorProb_ge_one_sub_budget_mul_massBound
PRB4 ObserverModel.vanishingError
PRB5 ObserverModel.not_vanishingError_of_lower_bound
PRB6 ObserverModel.family_not_vanishing_of_budget_mass_gap
PRB7 ObserverModel.errorProb_uniform_ge_one_sub_budget_div_card
PRB8 ObserverModel.successProb_uniform_le_budget_div_card
PRB9 ObserverModel.weak_fano_uniform_budget_lower_bound
PRB10 ObserverModel.uniform_family_not_vanishing_of_budget_ratio_gap
PRB11 ObserverModel.successProb_uniform_eq_card_div_card
PRB12 ObserverModel.errorProb_uniform_eq_one_sub_card_div_card
PRB13 ObserverModel.weak_fano_uniform_successSet_lower_bound
PRB14 ObserverModel.weak_fano_uniform_via_successSet
PRB15 ObserverModel.uniform_success_failure_partition_entropy
PRB16 ObserverModel.fano_uniform_budgeted
PRB17 ObserverModel.fano_uniform_observation_only
PRB18 ObserverModel.observation_only_successProb_uniform_le_one_div_card
PRB19 ObserverModel.observation_only_errorProb_uniform_ge_one_sub_one_div_card
PRB20 ObserverModel.successProb_le_budget_mul_maxMass
PRB21 ObserverModel.errorProb_ge_one_sub_budget_mul_maxMass
PRB22 ObserverModel.weak_fano_maxMass_lower_bound
PRB23 ObserverModel.family_not_vanishing_of_budget_maxMass_gap
PRB24 ObserverModel.successSetCard_ge_successProb_div_maxMass
PRB25 ObserverModel.successSetCard_ge_one_sub_error_over_maxMass
PRB26 ObserverModel.observation_only_successProb_le_maxMass
PRB27 ObserverModel.observation_only_errorProb_ge_one_sub_maxMass
PRB28 ObserverModel.successProb_le_budget_times_exp_neg_minEntropy
PRB29 ObserverModel.errorProb_ge_one_sub_budget_times_exp_neg_minEntropy
PRB30 ObserverModel.sourceEntropy_uniformFiniteSource
PRB31 ObserverModel.minEntropy_le_sourceEntropy
PRB32 ObserverModel.successSetCard_ge_one_sub_error_times_exp_minEntropy
PRB33 ObserverModel.budget_ge_one_sub_error_times_exp_minEntropy
PRB34 ObserverModel.inv_card_le_maxMass
PRB35 ObserverModel.minEntropy_le_log_card
PRB36 ObserverModel.observation_only_successProb_le_exp_neg_minEntropy
PRB37 ObserverModel.observation_only_errorProb_ge_one_sub_exp_neg_minEntropy
PRB38 ObserverModel.sourceEntropy_nonneg
PRB39 ObserverModel.sourceEntropy_le_log_card
PRB40 ObserverModel.minEntropy_le_log_budget_sub_log_one_sub_error
PRB41 ObserverModel.observation_only_minEntropy_le_neg_log_one_sub_error
PRB42 ObserverModel.sourceEntropy_le_success_failure_partition
PRB43 ObserverModel.fano_arbitrary_budgeted
PRB44 ObserverModel.fano_arbitrary_observation_only
PRB45 ObserverModel.fano_arbitrary_conditional_style
PRB46 ObserverModel.fano_arbitrary_conditional_observation_only
PRB47 ObserverModel.sourceEntropy_eq_observationTagEntropy_add_conditionalEntropyGivenPair
PRB48 ObserverModel.conditionalEntropyGivenPair_le_sourceEntropy
PRB49 ObserverModel.mutualInfoSurrogate_eq_observationTagEntropy
PRB50 ObserverModel.conditionalEntropyGivenPair_le_fano_arbitrary
PRB51 ObserverModel.conditionalEntropyGivenPair_le_fano_observation_only
PRB52 ObserverModel.observationTagEntropy_le_log_budget
PRB53 ObserverModel.mutualInfoSurrogate_le_log_budget
PRB54 ObserverModel.exactOn_clique_subsetEntropy_le_log_tags
PRB55 ObserverModel.successSet_clique_entropy_le_log_tags
PRB56 ObserverModel.pmfEntropy_successFailurePMF
PRB57 ObserverModel.mutualInfoDeterministic_eq_observationTagEntropy
PRB58 ObserverModel.mutualInfoDeterministic_eq_source_minus_conditional
PRB59 ObserverModel.mutualInfoDeterministic_nonneg
PRB60 ObserverModel.mutualInfoDeterministic_le_log_budget
PRB61 ObserverModel.conditionalEntropyGivenPair_nonneg
PRB62 ObserverModel.observationTagEntropy_le_sourceEntropy
PRB63 ObserverModel.mutualInfoDeterministic_le_sourceEntropy
PRB64 ObserverModel.coarsenedObservationEntropy_le_observationTagEntropy
PRB65 ObserverModel.coarsenedMutualInfoDeterministic_eq_coarsenedObservationEntropy
PRB66 ObserverModel.coarsenedMutualInfoDeterministic_le_original
PRB67 ObserverModel.deterministicKernel_data_processing
PRB68 ObserverModel.deterministicKernel_entropy_data_processing
PRB69 ObserverModel.inducedPairPMFFin_eq_uniform_of_klDiv_zero
PRB70 ObserverModel.observationTagEntropy_eq_log_budget_of_klDiv_zero_uniform
PRB71 ObserverModel.mutualInfoDeterministic_eq_log_budget_of_klDiv_zero_uniform
PRB72 ObserverModel.nonuniform_inducedPairPMFFin_implies_klDiv_ne_zero
PRB73 ObserverModel.observationTagEntropy_gap_nonneg
PRB74 ObserverModel.observationTagEntropy_gap_eq_zero_of_klDiv_zero_uniform
PRB75 ObserverModel.observationTagEntropy_gap_pos_implies_klDiv_ne_zero
PRB76 ObserverModel.mutualInfoDeterministic_gap_pos_implies_klDiv_ne_zero
PRB77 ObserverModel.successFailureFiniteSource_entropy_eq_binEntropy
PRB78 ObserverModel.successFailureEntropy_le_sourceEntropy
PRB79 ObserverModel.rvEntropy_failureRVFin_eq_binEntropy
PRB80 ObserverModel.rvEntropy_failureRVFin_le_sourceEntropy
PRB81 ObserverModel.decodedOutputEntropy_le_mutualInfoDeterministic
PRB82 ObserverModel.decodedOutputEntropy_source_gap_nonneg
PRB83 ObserverModel.decodedOutputEntropy_log_gap_nonneg
PRB84 ObserverModel.decodedOutputEntropy_le_success_failure_partition
PRB85 ObserverModel.decodedOutputEntropy_fano_budgeted
PRB86 ObserverModel.observableEntropy_failureObservable_eq_binEntropy
PRB87 ObserverModel.decodedOutputEntropy_fano_observation_only
PRB88 ObserverModel.observableEntropy_failureObservable_le_sourceEntropy
PRB89 ObserverModel.decodedOutputObservable_eq_coarsenedObservationObservable
PRB90 ObserverModel.DeterministicObservable.entropy_coarsen_le_sourceEntropy
PRB91 ObserverModel.inducedDecodedOutputPMFFin_eq_uniform_of_klDiv_zero
PRB92 ObserverModel.decodedOutputEntropy_eq_log_outputAlphabet_of_klDiv_zero_uniform
PRB93 ObserverModel.decodedOutputEntropy_gap_eq_zero_of_klDiv_zero_uniform
PRB94 ObserverModel.decodedOutputEntropy_gap_pos_implies_klDiv_ne_zero
PRB95 ObserverModel.decodedOutputObservable_entropy_le_sourceEntropy
UNQ1 uniqueness